VeriGate: An Adaptive, Self-Verifying Semantic Cache for Scalable and Cost-Efficient LLM Gateways

A report submitted for the course named Project - II (CS4201)

Submitted By

ABHISHEK YADAV
SEMESTER - VII
230103041

Supervised By

DR. SALAM MICHAEL SINGH



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

NOVEMBER, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

ABHISHEK YADAV
230103041

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Salam Michael Singh
Assistant Professor

Email: michael_s@iiitmanipur.ac.in
Contact No: +91 8575443081

To Whom It May Concern

This is to certify that the project report entitled **"VeriGate: An Adaptive, Self-Verifying Semantic Cache for Scalable and Cost-Efficient LLM Gateways",** submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of work carried out by **ABHISHEK YADAV** bearing roll number 230103041.

Signature of Supervisor

(Dr. Salam Michael Singh)

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

VeriGate is an organization-aware LLM gateway for internal developer tools, technical support, and applications across separately configured organizations. Authorized employees share eligible workspace answers while private responses remain isolated. It addresses repeated technical requests while avoiding reuse of a nearby but unsuitable recorded answer. The gateway authenticates a principal, limits reuse to authorized workspace or private scope, retrieves exact or semantic candidates, and uses an optional answer-aware policy to assess the stored question, stored answer, and incoming task before a cache hit. Otherwise it calls the configured provider. Avoiding repeated generation is the intended cost benefit; gateway computation and storage still incur overhead. Self-verifying denotes this suitability assessment, not guaranteed factual truth. Exact matching remains the default. Functional tests, controlled answer labels, and saved MiniLM/RedisSearch HTTP runs evaluate different parts of the design. In a fresh 25-probe authored HTTP workload, combined technical guards served 8 of 15 valid opportunities versus 6 of 15 for the baseline, with no invalid accept among five examples; the improvement missed its predeclared promotion gate. Mock-provider timings do not establish billed savings or broad scalability. Independent answer-level review and real-provider validation remain necessary before production semantic reuse.

Acknowledgement

I would like to express my deepest appreciation to all those who provided me the possibility to complete this project. A special gratitude I give to my supervisor, **Dr. Salam Michael Singh**, whose contribution in stimulating suggestions and encouragement helped me to coordinate my project especially in writing this report.

I would also like to express my sincere gratitude to the Head of the Department (CSE), **Dr. Navanth Saharia**, for his constant support and for providing the necessary facilities to successfully execute this project.

Furthermore, I would also like to acknowledge with much appreciation the crucial role of the faculty and staff of the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur, who gave the permission to use all required equipment and necessary materials to complete the tasks.

Last but not least, many thanks go to my family and friends whose support, guidance, and encouragement have been invaluable throughout this endeavor.

ABHISHEK YADAV
Roll No: 230103041

Contents

Abstract **3**

Acknowledgement **4**

List of Figures **7**

List of Tables **8**

1 Introduction **9**
1.1 Background and Motivation . . . 9
1.2 Intended Users and Organizational Use . . . 9
1.3 Problem Definition . . . 10
1.4 Objectives and Scope . . . 10
1.5 Cost and Latency Objectives . . . 10
1.6 Development Record . . . 11

2 Literature Survey **12**
2.1 Caching and Verification . . . 12
2.2 Position of This Work . . . 12

3 Requirement Engineering **14**
3.1 Users, Boundary, and Assumptions . . . 14
3.2 Functional Requirements . . . 14
3.3 Quality Requirements and Limits . . . 14
3.4 Requirements Traceability . . . 15
3.5 Stakeholders and Operational Use Cases . . . 15
3.6 Inputs, Outputs, and Acceptance Criteria . . . 16
3.7 Nonfunctional Constraints and Threat Boundaries . . . 16
3.8 Traceability and Unresolved Release Criteria . . . 17
3.9 Numbered Requirements and Feasibility . . . 17

4 System Design **19**
4.1 Boundary, Notation, and Source Checkpoint . . . 19
4.2 Actors and Supported Goals . . . 19
4.3 Context Data Flow . . . 20
4.4 Level-1 Data Flow and Stores . . . 20
4.5 JSON Request Control Flow . . . 23
4.6 Generation, Completion, and Storage Control . . . 25
4.7 Candidate Assessment and Request Selection . . . 25
4.8 Semantic Hit Interaction . . . 27
4.9 JSON Miss and Fallback Interaction . . . 27
4.10 Organization, Workspace, and Private Identity . . . 27
4.11 Logical Redis Storage and Index . . . 28
4.12 Local Deployment and Process State . . . 29
4.13 Startup, Readiness, and Cache Residence . . . 31

5 Implementation **32**
5.1 Gateway Request Path . . . 32

5.2 Retrieval and Candidate Selection . . . 32
5.3 Credentials and Automatic Scope Resolution . . . 33
5.4 Organization Example . . . 33
5.5 Providers, Storage, and Concurrency . . . 33
5.6 Configuration Profiles and Observability . . . 34
5.7 Experimental Task Verifier . . . 34
5.8 Source Map and Runtime Ownership . . . 34
5.9 Startup, Readiness, and Local Versus Network Work . . . 35
5.10 Request Schema and Authentication . . . 36
5.11 Embedding and Scoped Retrieval . . . 36
5.12 Answer Suitability and Selection . . . 37
5.13 Provider Completion, Retention, and Errors . . . 37
5.14 Four Request Walkthroughs . . . 38

6 Results and Evaluation **39**
6.1 Evidence Types and Metrics . . . 39
6.2 Functional and Integration Verification . . . 39
6.3 Historical Query-Pair and Proxy Experiments . . . 39
6.4 Judge Validation and Public-Query Pilots . . . 40
6.5 Controlled Answer-Aware Evaluations . . . 40
6.6 Real Retrieval and HTTP Path . . . 41
6.7 Latency and Resource Measurements . . . 41
6.8 When Caching Can Reduce Cost . . . 42
6.9 Follow-up Audit Fixes and Controlled HTTP Demonstration . . . 42
6.10 Limitations and Threats to Validity . . . 44
6.11 Testing Strategy and Execution Layers . . . 45
6.12 Representative Functional and Failure Cases . . . 46
6.13 Semantic Evaluation Protocol . . . 46
6.14 The 1,000-Request Stress Study . . . 46
6.15 Positive, Negative, and Resource Findings . . . 48
6.16 Latency and Conditional Cost Model . . . 49
6.17 Testing Summary and Remaining Gates . . . 49

7 Conclusion **50**
7.1 Conclusion . . . 50
7.2 Future Work . . . 50

A Historical Proxy Tables **51**

Bibliography **53**

List of Figures

1.1 Indicative project timeline by phase, August to November 2026. Each bar spans the weeks in which that phase was the main focus; phases overlap, and report writing runs throughout. The final phase is the production-readiness audit followed by the evaluation-integrity, continuous-integration, and report reconciliation work described in Chapter 6. 11

4.1 Level-0 context DFD with separate data flows in each direction. 20

4.2 Level-1 DFD: authorized retrieval, generation, storage, response, and operations. . . 22

4.3 JSON request admission, scoped lookup, and HIT decision (A/B connectors). 24

4.4 Candidate action precedes top-k selection and final request outcome. 26

4.5 Compose containers, shared Redis state, and optional external HTTP. 30

6.1 Evaluation provenance. Development examples are diagnosed separately; frozen authored labels remain a limited substitute for independent real-user review. 41

6.2 Dashboard result panel for a controlled-evaluation fixture request, captured after the answer-source correction. The always-visible *answer source* reads "controlled evaluation fixture" (never "generated"); the expanded "How this request was handled" panel shows the actual provider, the workspace lookup scope, the `stored` write outcome, and that the verifier was not invoked on the exact-shortcut path. A labelled development fixture shown to illustrate the corrected provenance labelling and the evidence the dashboard surfaces, not generated-answer, accuracy, or cost evidence. . 44

List of Tables

2.1 Literature methods and VeriGate's implemented scope. Published outcomes are not VeriGate experiments. . . . 13

3.1 Requirements, implementation, and available verification at the fixed source checkpoint. . . . 15

5.1 Representative configurations; flags are independent choices, not one combined production mode. . . . 34

5.2 Code-grounded module map at the recorded source checkpoint. . . . 35

6.1 Distinct HTTP workloads with controlled provider. Counts are per frozen workload; rows must not be pooled. . . . 41

6.2 Testing layers and what their evidence can establish. . . . 45

6.3 Representative test cases, part 1. . . . 47

6.4 Representative test cases, part 2. . . . 47

A.1 Candidate-pair smoke fixture. Positive recall and negative acceptance use different denominators; this is not workload hit rate or monetary savings. . . . 51

A.2 Five-seed query-pair proxy transfer; false acceptance is among accepted pairs, not answers. . . . 51

A.3 Chronological QQP pair-proxy replay; unknown cross-pair outcomes are not counted as correct. . . . 51

A.4 Synthetic IID bound exercise; reject-all runs have no defined conditional risk. . . . 52

A.5 Recorded stage timings in milliseconds; these are not end-to-end provider latency. . . 52

A.6 Recorded lookup p50 latency in milliseconds; the crossover size was not measured. . 52

A.7 Illustrative cascade cost model. Quality retention is hard-label recall, not judged answer quality. . . . 52

Chapter 1

Introduction

1.1 Background and Motivation

Internal developer tools, technical support systems, and organizational applications often receive repeated technical questions. An authorized employee may ask how to inspect a deployment; another employee in the same workspace may ask the same task in different words. Reusing a stored response can avoid a provider call. Yet a nearby question can request a different action, environment, permission, or software version. The nearest question is therefore only a *candidate* for reuse; its recorded answer must also fit the new task.

VeriGate is an organization-aware LLM gateway for applications that explicitly integrate with its supported HTTP API. It is not a chatbot. A normal ChatGPT website or Codex session does not automatically pass through this gateway, and a user's subscription credential is not extracted for provider access. An application sends a VeriGate credential; the server resolves its provider profile and owns the upstream provider credential.

1.2 Intended Users and Organizational Use

The intended installation is one gateway service that can serve several organizations while keeping their data separate. Within an organization, an administrator configures trusted applications, employee identities, workspace membership, and provider profiles. Employees use their existing integrated application; they need not visit a gateway administration screen or choose a storage scope for every question. When the request omits a scope override, the server resolves an authorized default and may restrict storage further based on the request and response.

Consider an engineering workspace in Organization A. One employee asks how to inspect a deployment and receives a reusable procedure. A colleague later asks for the same procedure in different words. If the stored answer satisfies the colleague's task and the sharing rules permit access, the gateway can reuse it. A private account-specific response must not enter that shared pool. An employee in Organization B cannot retrieve Organization A's entry even when the question is identical. Organization B builds its own eligible cache. Thus the intended savings arise from repeated work across authorized users, not from asking one user to repeat a question many times.

This model is suitable for internal developer portals and technical support applications with recurring procedures. Requests requiring fresh observations, private account state, or unsupported conversation history need different handling. Multi-organization isolation is an implemented

configuration boundary; operating at thousands of organizations remains a capacity-validation goal.

1.3 Problem Definition

The gateway must distinguish four operations. An *exact repeat* can use a scope-local stored response after a current-entry check. *Semantic retrieval* embeds a new request and returns nearby stored questions from an authorized scope; similarity only ranks candidates. *Answer-aware assessment* compares the incoming question with a stored question *and its particular answer*, using structural checks and an optional model signal. If no candidate is approved, *provider fallback* generates a new response. A candidate rejected or marked VERIFY has not thereby been proved false; it is simply not served from cache.

The engineering problem includes access control as well as quality. Employees A and C may share eligible technical answers in a workspace when both are authorized. Private answers remain restricted to the subject and application scope. A content classifier can conservatively narrow a shared write, but it cannot establish that all sensitive content has been detected.

1.4 Objectives and Scope

The implemented prototype provides credential resolution, organization and provider-profile configuration, exact or semantic cache modes, scoped Redis storage, a local MiniLM embedding option, RedisSearch top-k retrieval, optional answer-aware candidate assessment, provider routing, JSON and streaming responses, admission controls, and operational metrics. Exact mode is the general default. The historical adaptive threshold and query-only verifier are alternative baseline paths when the answer-policy artifact is absent; they do not run simultaneously with the answer-aware policy.

The evaluation asks whether specific requests follow the intended scope and decision path and how much suitable reuse the opt-in policy achieves on controlled examples. It does not establish factual correctness of arbitrary upstream answers, production monetary savings, a universal latency target, or company-scale capacity. The contribution is the integration of scope-aware storage with candidate-level answer suitability checks and a traceable evaluation of their trade-offs, including negative results. Related caches already support semantic search, adaptive boundaries, and verification (Chapter 2).

1.5 Cost and Latency Objectives

The economic motivation is to avoid generating the same useful answer repeatedly. On a cache hit, VeriGate avoids the upstream generation call. It still pays for its own CPU, memory, Redis operations, and any verification. On a miss, those checks can add work before generation. Cost effectiveness therefore depends on how often eligible requests recur, how reliably suitable answers are accepted, and the relative cost of verification and generation. Chapter 6 gives a break-even model alongside measured results rather than assuming that every cache hit is free.

1.6 Development Record

The repository contains versioned development, controlled-evaluation, and live-path checkpoints. Figure 1.1 summarizes how the work progressed by phase from August to November 2026, from the first gateway and query-pair studies, through the answer-aware policy and the semantic retrieval and HTTP measurements, to the organization-sharing experiments and the late-October production-readiness audit and the evaluation-integrity, continuous-integration, and report follow-up that closed the period. The chart is indicative and organized by phase rather than claiming independently verified daily effort; each measured experiment still carries its own recorded source revision.

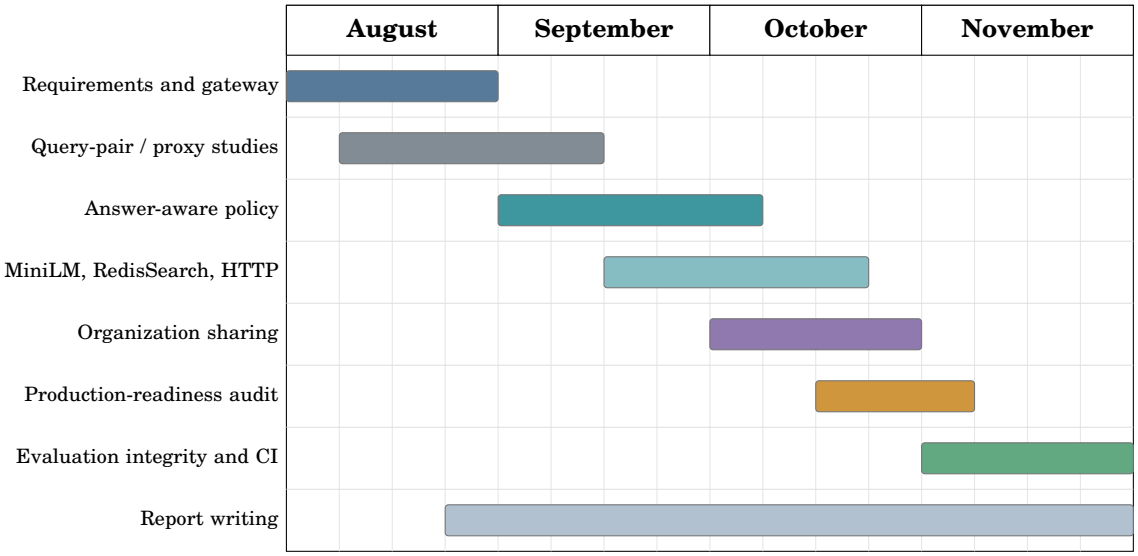


Figure 1.1: Indicative project timeline by phase, August to November 2026. Each bar spans the weeks in which that phase was the main focus; phases overlap, and report writing runs throughout. The final phase is the production-readiness audit followed by the evaluation-integrity, continuous-integration, and report reconciliation work described in Chapter 6.

A twelve-week historical schedule is not independently evidenced, so this report uses the recorded phases: initial gateway and pair-proxy work; answer-aware policy and controlled examples; RedisSearch and HTTP-path measurements; organization sharing and technical applicability experiments. Chapter 6 identifies each run's source, label type, and measurement boundary. The main implementation narrative records checkpoint `37dc7a7`, with diagram and dashboard reconciliation through `1cdb80a`. Each measured experiment retains its own source revision. Later changes can update the implementation record without rewriting these historical observations.

A later release should record its commit, enabled profile and test evidence separately. Configuration and model changes update the implementation description; they do not retroactively change the dataset, timing boundary or outcomes of a historical experiment.

Chapter 2

Literature Survey

2.1 Caching and Verification

Exact caching uses request identity as its reuse condition, in the spirit of classic lexical retrieval that scores documents by weighted term overlap [18]. Semantic caching instead retrieves neighbors in an embedding space and adds a decision rule. Transformer language models [21, 4] underpin both the dense sentence embeddings that make related requests easier to retrieve [16] and the compact distilled encoders, such as MiniLM, that keep that step affordable [22]. Approximate nearest-neighbor indexes such as HNSW [10], product quantization [8], and GPU-accelerated libraries [9] can search large vector collections, but none of them decides whether a recorded answer satisfies the later request. That decision is closer to natural-language inference, whose benchmark corpora [2] and strong encoders [7] VeriGate reuses for its answer-assertion check. Verification can act on the two questions, on a stored answer and incoming task, or on a newly generated response; these evaluate different objects and incur different latency and provider cost.

Cost-aware LLM serving is an active area in its own right: cascades that escalate only hard queries to an expensive model [3] and learned request routers [12] both trade quality against spend, and recent surveys catalogue the broader efficient-inference design space [23]. Caching is complementary to these, avoiding the provider call entirely when a suitable answer already exists.

GPTCache provides configurable embedding, storage, similarity, and post-processing components rather than a single mandatory fixed threshold [1, 6]. vCache learns cache-entry-specific boundaries online and reports a user-specified error-rate bound under its paper's modelling and feedback assumptions [19]. Its published benchmarks and guarantee are not reproduced by VeriGate. Krites combines static and dynamic tiers; a near miss can trigger an asynchronous LLM judge and later promotion without putting that judge on the current request's critical path [20]. This differs from VeriGate's synchronous, optional candidate assessment. These systems make it inaccurate to claim that existing caches uniformly trust similarity alone.

2.2 Position of This Work

VeriGate's engineering contribution is the explicit authorized scope, provider-profile compatibility, final candidate identity recheck, and an answer-aware selection path with visible abstentions. Its research contribution is a bounded comparison of baseline and experimental policies on labelled technical tasks and real retrieval paths. It does not introduce semantic caching, adaptive thresholds, or verification as general ideas. A zero-error count on a small selected set is descriptive and is not a formal risk guarantee. Query-pair labels test paraphrase decisions; answer-level labels test whether one stored answer fits a new request. Chapter 6 keeps those denominators separate.

Table 2.1: Literature methods and VeriGate's implemented scope. Published outcomes are not VeriGate experiments.

System	Candidate boundary	Suitability signal	Cost and evidence boundary
GPTCache	Configurable similarity evaluator	Optional post-processor	Depends on configured pipeline
vCache	Learned entry-specific boundary	Online response comparison	Published bound under stated assumptions
Krites	Static threshold plus grey zone	Asynchronous LLM judge	Judge cost moves off current request path
VeriGate	Exact default; optional top-k semantic mode	Stored question, answer, incoming question; checks and pinned NLI	Synchronous verifier latency; controlled evidence only

Chapter 3

Requirement Engineering

3.1 Users, Boundary, and Assumptions

The direct client is an internal developer, support, or organizational application using the gateway's supported chat endpoints. Its VeriGate credential authenticates access and resolves a principal: organization, subject, application, workspaces, permissions, and provider profile. A provider credential is configured on the server for that profile. Authorized employees can reuse eligible workspace answers; private content is scoped to the subject and application. This is an application integration boundary, not an automatic route for consumer ChatGPT or Codex sessions.

The JSON request contract accepts a prompt or supported message structure, an optional cache scope and workspace, bypass, and a supported streaming route. Request-body, prompt, and response limits are configured in the gateway; the implementation is not a universal drop-in OpenAI proxy. Invalid credentials, unauthorized scopes, admission failures, deadlines, and upstream errors produce explicit HTTP errors. Development mock and controlled-evaluation providers are selected configurations. A live provider failure may use configured provider failover; it does not automatically turn into a successful mock answer.

3.2 Functional Requirements

The gateway shall authenticate a principal before cache access; resolve scope and provider compatibility; bypass volatile or ineligible requests; serve current exact entries; rank eligible semantic candidates when semantic mode is enabled; assess the stored answer before a semantic hit when an answer policy is selected; call a configured provider on a miss or bypass; and store only eligible completed responses within the authorized scope. An explicit private request must never broaden into workspace sharing. A workspace request without permission must fail or narrow according to the documented scope decision.

3.3 Quality Requirements and Limits

The desired outcome is fewer upstream calls at acceptable answer suitability and bounded lookup latency. These are goals, not blanket guarantees. A cache lookup consumes Redis, embedding, search, and possibly verifier time; a process-local vectorized scan grows with the number of entries. TTL bounds residence time, not factual freshness. LRU and configured per-scope quotas manage entries; a daily admitted-miss count does not cap billed tokens or money. A rejected candidate is

an abstention or policy decision, not a proven false hit. Observed latencies and suitability counts appear only with their measurement boundaries in Chapter 6.

3.4 Requirements Traceability

Table 3.1: Requirements, implementation, and available verification at the fixed source checkpoint.

Requirement	Component	Evidence	Status
Credential and scoped sharing	`organization.py`, `main.py`	Organization-scope tests; saved mock HTTP observations	Tested locally
Exact cache and retention	`semantic_cache.py`, `storage.py`	Gateway and capacity tests	Tested locally
Semantic shortlist	MiniLM, RedisSearch store	Live Redis/MiniLM run and shortlist traces	Opt-in
Answer suitability	`answer_policy.py`	Frozen controlled and HTTP workloads	Opt-in
Completed provider response	Provider registry and gateway	Completion, deadline, streaming tests	Tested locally
Production answer quality	Independent reviewed workload	No sufficient labelled live workload	Open
Scale and monetary savings	Load and billed-provider study	No production-scale validation	Open

3.5 Stakeholders and Operational Use Cases

The client application supplies a prompt and receives either a previously completed answer, a newly generated answer, or an explicit error. The developer integrating that client needs a stable single-turn API, visible outcome metadata, and a way to request private or no-store treatment. An organization administrator needs control over credential membership, workspace permission, provider profiles, and retention limits. An operator needs readiness, counters, and failure reasons to tell an empty cache from an unhealthy verifier or provider. The person whose content appears in a request needs the gateway to keep private material outside shared workspace entries. These interests constrain the same request path; optimizing hit rate alone would not satisfy them.

The primary use case begins when an authenticated application submits a supported request. The gateway resolves the principal and provider route, classifies the requested scope, and checks whether the text can be cached. If an exact answer exists under that identity, it may return it after a current-entry check. Otherwise semantic mode may retrieve a bounded shortlist and assess each stored answer against the new task. An accepted and still-current candidate gives a HIT. If the shortlist is empty, rejected, uncertain, conflicting, or over budget, the gateway calls the configured provider. A normally completed JSON response is considered for storage only if retention policy permits, then returned with its storage outcome. Provider or admission failure produces an error rather than an invented cached answer.

A second use case is authorized collaboration. Alice asks a reusable workspace question, and Carol later asks an equivalent question as another member of that same workspace. The answer may be reused only when Alice's content was eligible for shared storage, Carol has permission, and the provider compatibility identity agrees. In the selected isolated HTTP probe, a seeded answer was served to Carol; another valid paraphrase of Alice's completed response missed despite being

in scope. This distinction makes workspace authorization a precondition, not a promise that all semantically related requests will hit.

A third use case is privacy or uncertainty. An explicit private request must remain in a subject/application scope. A secret-bearing or volatile request can bypass storage. A changed technical action or target should decline an old answer and let the provider handle the new task. If a verifier cannot decide within its token or time bound, the safe outcome is a MISS. This can increase cost and latency; the report measures such lost opportunities rather than hiding them inside an overall success count.

3.6 Inputs, Outputs, and Acceptance Criteria

The application supplies a bounded JSON body, a gateway credential, and optionally a scope request. A prompt or one supported user message is required. A workspace identifier is meaningful only after the resolved principal proves membership and permission. Extra fields are rejected by the current schema rather than silently forwarded. A successful JSON response identifies the outcome and includes generated or reused content; a streaming response has content events followed by completion metadata only after normal termination. Metrics and candidate traces are operational evidence, not a substitute for client-visible correctness.

Acceptance of a cached answer has three separate conditions. First, the principal and requested scope must authorize access to that entry. Second, the candidate must still be present and unexpired when selected. Third, a semantic candidate must pass the selected suitability policy; high vector similarity alone is insufficient. The exact shortcut uses request identity and entry checks rather than running NLI. A candidate may pass retrieval and fail suitability, as in the saved live challenges. Conversely, a policy might approve an unsuitable candidate on an authored test. The test plan therefore records retrieval inclusion, policy action, request HIT/MISS, and independent label separately.

For a newly generated answer, successful completion requires a configured provider route to return a normal terminal signal within its limits. The gateway can deliver a completed answer even if a later cache write fails, while reporting that failure. It must not cache an incomplete stream or an upstream error. For an authorized workspace request, the response may be narrowed to private storage if the content classification requires it. These are observable behaviors that can be asserted with local mock providers; they do not establish production answer quality for a real external provider.

3.7 Nonfunctional Constraints and Threat Boundaries

Latency has multiple budgets. Authentication and admission precede the gateway timer reported in a JSON response. Cache lookup can include Redis reads, a local embedding, ANN search, and NLI inference. A miss additionally pays the configured provider's latency. The selector limits candidate count and verification time to avoid letting an unusually dense shortlist consume an unbounded critical path. A finite limit can convert a potentially reusable answer into a MISS; this is a documented trade-off rather than an invisible timeout.

Scope separation is enforced through server-resolved identity and an opaque Redis tag. An attacker with an application credential must not gain another workspace by submitting a workspace field. A same-worded question in another organization must use a different cache namespace. The content classifier is a defense for obvious private markers, but arbitrary confidential text may evade heuristics. The deployment therefore still depends on correct client classification, credential

governance, Redis protection, and appropriate retention settings. The local tests cover selected paths, not a complete adversarial privacy assessment.

Capacity limits include entry count, bytes, TTL, and least-recently-used eviction at scope and organization levels. These protect finite storage but can lower hit rate under pressure. Multiple gateway workers share Redis state, while each holds its own models and local coalescing state. Memory scales with worker count; duplicate provider calls across workers remain possible. A daily admitted-miss counter is an operational guard and cannot be interpreted as a billed-cost ceiling because token lengths and provider prices vary.

3.8 Traceability and Unresolved Release Criteria

The source map in Chapter 5 and the case catalogue in Chapter 6 connect requirements to implementation and evidence. Authentication and scope have test assertions and selected HTTP observations. Semantic suitability has labelled development and frozen challenges, with known valid misses and some unsafe approvals. Provider completion and storage failure have injected-failure tests. The dashboard's final browser behavior, exact-final-revision full suite, independently reviewed answer labels, real-provider billing, and target-scale operation remain release criteria. A historical pass count cannot close a criterion after runtime code changes, and a controlled provider cannot establish external provider economics.

3.9 Numbered Requirements and Feasibility

The requirements below use "shall" for implemented contract behavior at the recorded checkpoint. They do not state a numeric production service-level objective. The table in the preceding section maps major requirements to components and saved checks.

FR1. The gateway shall validate a single-turn request and authenticate a gateway credential before accessing scoped cache content.

FR2. It shall resolve organization, subject, application, workspace permission, and provider profile on the server; an override shall not grant membership.

FR3. It shall search only entries under a compatible scope and configured policy, with an exact shortcut and an optional bounded semantic shortlist.

FR4. When answer-aware mode is enabled, it shall assess the stored answer for the incoming task and abstain on unsupported, conflicting, or over-budget candidates.

FR5. It shall call an explicitly configured provider on a miss or bypass and require a normal complete response before considering storage.

FR6. It shall honor no-store and private restrictions, TTL and quota limits, and shall not turn a cache-write failure into loss of a completed provider answer.

NFR1. Lookup work shall have finite candidate and time bounds, and request/provider deadlines shall yield explicit outcomes.

NFR2. Shared state shall be scoped and observable through outcome metadata, timing and reason traces without exposing provider secrets in cache keys.

NFR3. Readiness shall depend on configured required components; multiple workers shall not assume process-local coalescing is shared.

Technical feasibility is supported by the local Compose deployment, the saved Redis Stack and model-backed HTTP probe, and bounded model resource measurements. It does not establish the capacity of a production tenant population. The pinned NLI model's observed resident-memory increase of about 681 MB in one process makes worker count a deployment constraint. Redis Stack must be provisioned with a compatible vector index and finite memory; a semantic profile also needs local model artifacts. The mock profile permits repeatable integration without paid calls, while an external provider profile needs a network route and server-held credentials. Economic feasibility remains conditional: expected provider generation avoided by suitable HITs must exceed gateway inference, Redis, hosting, and operational cost. The stress study counts controlled-provider calls but provides no billed token or infrastructure totals, so the break-even inequality in Chapter 6 is a decision model rather than a savings result.

Chapter 4

System Design

4.1 Boundary, Notation, and Source Checkpoint

The diagrams in this chapter describe the implemented gateway as inspected at source commit `1cdb80a78ebb8bd00a53cea42aa080a523af950e`. They abstract selected paths rather than asserting that every script in the repository was audited. Concurrent application changes after this checkpoint need reconciliation before the drawings are used as a release record. The client is an integrated application acting for an authorized principal; the gateway does not intercept an ordinary ChatGPT or Codex session. Gateway credentials identify the caller, while server-configured provider profiles supply any upstream secret.

Figures 4.1 and 4.2 use the Yourdon/DeMarco data-flow convention: circles are numbered transformations, rounded rectangles are external entities, open-ended parallel lines are logical data stores, and labelled one-way arrows carry data. A DFD explains what crosses a boundary, not the order of conditionals. The flowcharts (Figures 4.3 and 4.4) use terminators, process rectangles, decision diamonds, and labelled yes/no branches, and the deployment drawing (Figure 4.5) uses container boundaries and communication links. These notations answer different questions and are not interchangeable. Shape carries the notation and colour reinforces the outcome: blue for a process, amber diamonds for a decision, teal for an interaction or alternate path, green for a successful terminator, red for an error or deny exit, and slate for an external entity or store. Shape and text retain their meaning in grayscale; colour is only a reading aid.

4.2 Actors and Supported Goals

Two actor roles drive the gateway directly. A client requests a technical answer and receives the outcome as part of that request; receiving an HTTP response is not a separate business use case. An operator can inspect occupancy or health/metrics data and can clear the caller's own private cache through supported routes. The cache-clear route does not erase an authorized workspace's shared answers for a normal user. The metric route uses its token dependency; the administration routes require the gateway credential. Some operational endpoints are also useful to automated deployment probes rather than a human operator.

4.3 Context Data Flow

Figure 4.1 has exactly one process, "0 VeriGate LLM Gateway." The client sends a gateway credential and prompt and receives an answer with outcome metadata or an error. In an external-provider configuration, the gateway sends a generation request and receives provider events, usage, or an error. The local mock profile stays inside the gateway boundary and therefore does not use the external arrow. Redis and local MiniLM/NLI inference are internal implementation components at this context level.

The operator flow represents concrete routes: `/admin/cache/clear`, `/admin/cache/occupancy`, `/admin/config`, `/metrics`, `/health`, `/ready`, and `/live`, each with its implemented access rule. An administrator does not push runtime configuration into the gateway through a depicted management API. Settings, provider secrets, and the optional organization directory are deployment inputs, discussed in Figure 4.5.

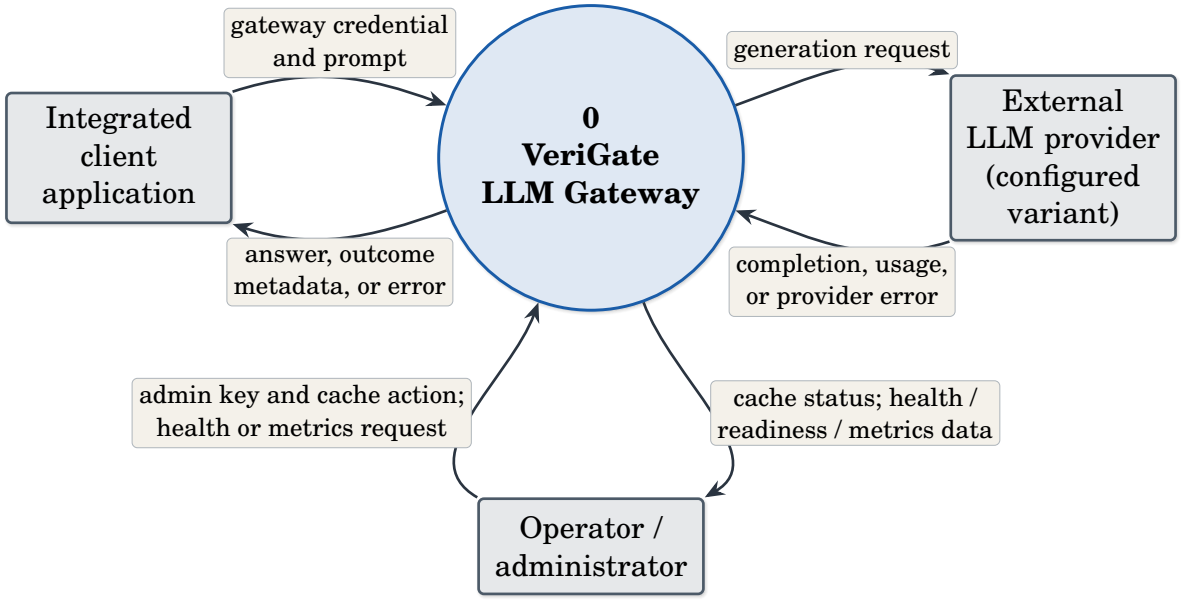


Figure 4.1: Level-0 context DFD with separate data flows in each direction.

4.4 Level-1 Data Flow and Stores

Figure 4.2 expands process 0 into eight logical transformations. Process 1.0 checks the request and gateway credential using D1's credential data. Process 2.0 resolves the authorized principal, compatible provider profile, workspace/private scope, and request eligibility using the same directory. Process 3.0 asks D2 for scoped exact or vector candidates and receives current candidate records. Process 4.0 assesses whether a stored answer fits the incoming task, returning a selected cached answer or a reason to generate. Process 5.0 applies admitted-miss control and obtains configured provider events. Process 6.0 validates the completed answer, classifies storage eligibility, and writes an eligible record to D2. Process 7.0 assembles content and outcome metadata. Process 8.0 represents the operational routes so operator data does not enter a client-response process by implication.

D1 is configuration loaded into the process, D2 is the scoped cache and optional RedisSearch vector index, D3 holds admitted-miss and quota state, and D4 is local policy/model material. D2 and D3 are logical stores that share Redis in the Redis-backed profile; they are not separate database

servers. D4 is per worker. An arrow carrying a candidate list is data, while a cache HIT or MISS is a later request outcome. The diagram abstracts the private overlay, index overfetch, failure exits, and atomic Redis transactions; the flowcharts and text below locate those details.

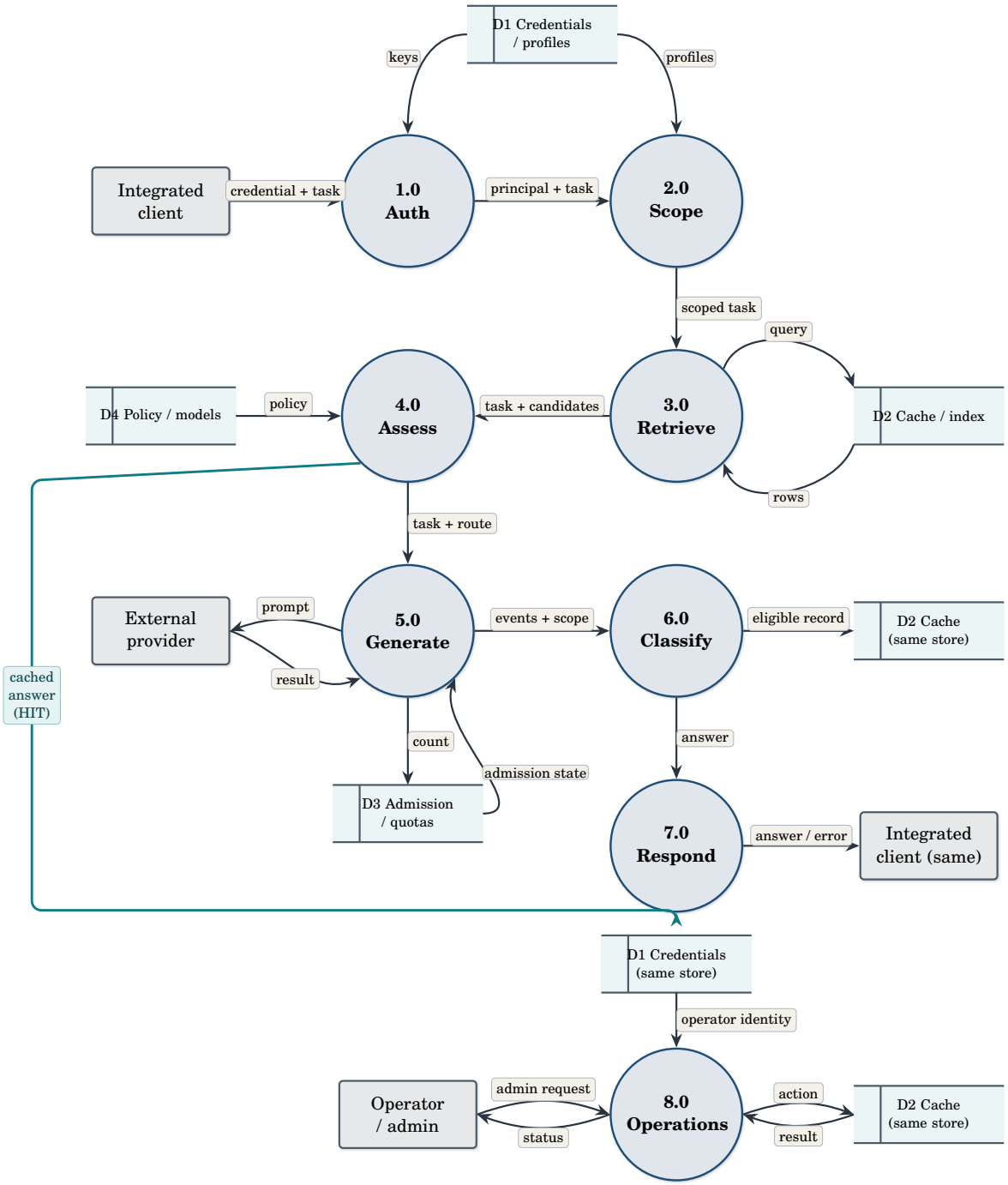


Figure 4.2: Level-1 DFD: authorized retrieval, generation, storage, response, and operations.

4.5 JSON Request Control Flow

Figure 4.3 describes the JSON handler's admission and lookup path. Request middleware limits body size and concurrency; schema validation and gateway authentication must succeed before cache work. The handler resolves a principal and registry, checks the rate limit, builds the prompt, and resolves an authorized scope. A no-store or explicit bypass request goes directly to generation. For a workspace request the private overlay is checked first; the scoped exact key is checked before any embedding. If semantic mode is active and exact lookup misses, MiniLM embeds the question and the selected backend retrieves a scope-filtered top-k set. An empty shortlist, VERIFY, REJECT, conflict, verification limit, deadline, changed entry, or unavailable cache produces a MISS reason rather than a cached answer.

The candidate decision in Figure 4.4 is a subprocess of this lookup, not an extra request path. A selected semantic entry is re-read before a HIT and its recency is touched without extending the entry's TTL. The gateway then rechecks the principal before publishing the HIT. A credential change returns an authorization error. The B connector leads to generation, described in the next section: both BYPASS and an unsuccessful lookup require admitted-miss control before a provider call.

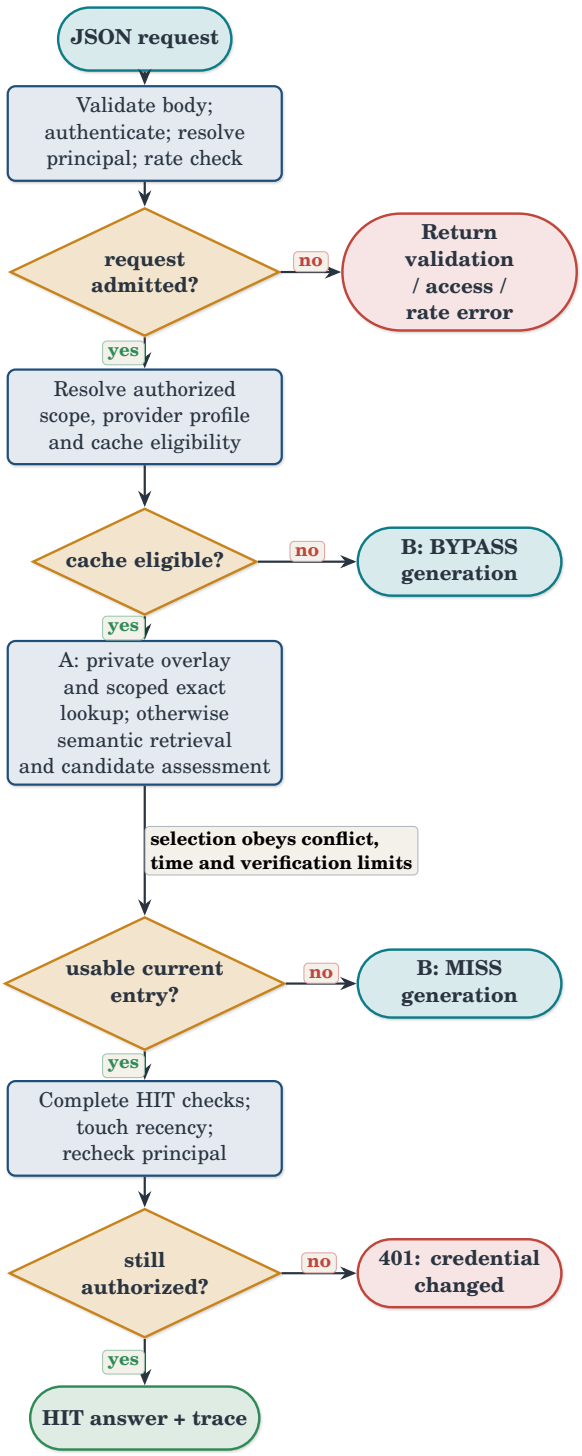


Figure 4.3: JSON request admission, scoped lookup, and HIT decision (A/B connectors).

4.6 Generation, Completion, and Storage Control

The gateway checks the principal again and counts one admitted miss or bypass. A rate rejection, unavailable admission state, or expired request deadline exits before a provider call. For a JSON MISS, a process-local singleflight key can let a concurrent follower receive the leader's completed answer; BYPASS generates directly. The provider registry selects an explicitly configured route and may use a configured alternate after a circuit-breaker or provider failure. It never silently substitutes the development mock for a real-provider failure.

The JSON generation function assembles content events and requires one normal terminal completion, a nonempty answer, and the configured response length bound. A timeout, incomplete stream, duplicate completion, or other provider error becomes an explicit error with no cache write. On success, the response classifier can mark the answer no-store, privately eligible, or shareable. A fallback route is not written under the intended route's identity. An eligible workspace response may be downgraded to the caller's private scope. A cache write may succeed, be refused by limits, time out, or fail; a completed answer is still returned with the recorded storage outcome, so a provider MISS and a non-success storage result remain distinct in the metadata. Streaming follows its own event/cancellation state machine and has no JSON singleflight guarantee.

4.7 Candidate Assessment and Request Selection

Figure 4.4 distinguishes a candidate action from the request outcome. The input is an incoming task, the stored question and answer, and the retrieval similarity. Structural checks can reject a changed number, polarity, named work, or unsupported answer type before model inference. Configured technical and applicability guards can also act on recognized actions, environments, and versions. These guards are optional; the experimental task-specific classifier is not a normal deployed verifier. Unresolved candidates go to the pinned answer-assertion NLI model. An unavailable model, overlong pair, uncertainty interval, or exhausted time budget produces VERIFY, an abstention rather than a judgement that the answer is false.

The cache coordinator assesses up to a configured limit and aggregates the resulting ACCEPT, VERIFY, and REJECT actions. A first ACCEPT does not guarantee a request HIT: accepted answers may conflict, a plausible lower-ranked candidate may remain unassessed, or the verification budget may expire. Only a selected answer whose Redis hash still matches the candidate may be served. The gateway also rechecks the principal before returning it. These checks trade some valid reuse for explicit uncertainty boundaries, as the measured valid misses in Chapter 6 show.

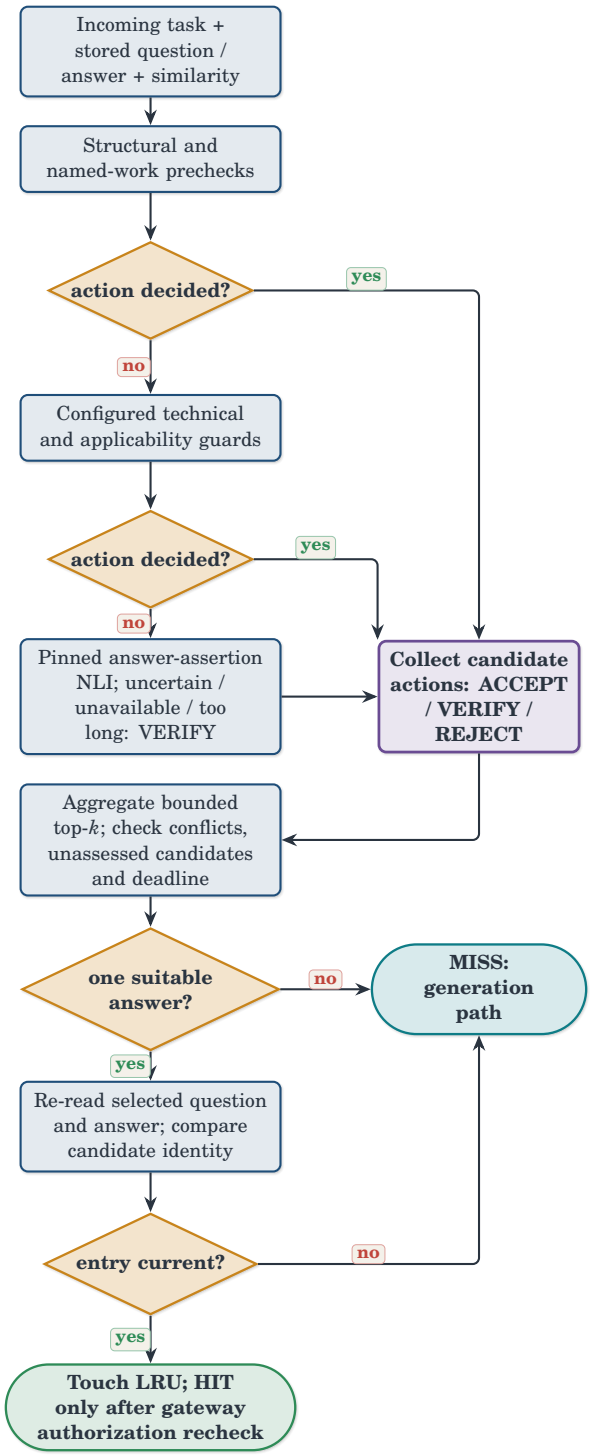


Figure 4.4: Candidate action precedes top-k selection and final request outcome.

4.8 Semantic Hit Interaction

The semantic HIT path runs only after an exact-key miss. The gateway has already authenticated the caller and resolved the authorized scope. The cache coordinator reads the scope-local exact key, receives no current entry, asks the local embedder for a normalized vector, and requests a tenant-filtered shortlist from RedisSearch. As Figure 4.4 shows, each eligible candidate can receive a precheck or verifier action, and the coordinator then checks conflicts and remaining plausible candidates.

For a selected candidate the coordinator re-reads the Redis hash and compares its stored question and answer. If the record has expired or changed, the request becomes a MISS. Otherwise a served semantic HIT updates LRU recency only; it does not renew the entry TTL. At this checkpoint the semantic identity read precedes the recency update; the operation is not an atomic read-and-serve transaction, so a subsequent mutation window remains. The cache coordinator returns the answer and trace to the gateway, which rechecks authorization and sends the client the HIT response. The provider has no role on this branch. The exact shortcut is described separately because it can return without embedding or NLI and touches recency before a confirming re-read in the current implementation.

4.9 JSON Miss and Fallback Interaction

The JSON MISS or BYPASS path proceeds as follows. The cache coordinator returns a reason: an empty scope, no suitable candidate, a verifier abstention, a conflict, or a stale selected entry. The gateway checks the principal again and charges an admitted miss before provider work. A rejected or unavailable admission result exits without generation. A JSON MISS can join process-local singleflight; a BYPASS calls the generator directly. A same-worker follower receives the leader's answer without its own provider call or cache write. Streaming does not use this JSON coalescing path.

The provider registry selects an intended configured route and prefetches the first provider event before it returns the selected provider name and a wrapped event stream. The gateway then iterates that stream to receive the remaining events. The JSON generator accumulates content and requires one normal terminal completion and a nonempty bounded answer. Timeout, provider error, or incomplete completion leads to an explicit error with no storage. A completed answer is classified against the request scope and provider route. A no-store answer, BYPASS, or successful alternate provider route skips storage under the intended cache identity. Eligible responses may be written to a shared workspace or downgraded to private scope. Storage success, refusal, timeout, and failure remain separate metadata outcomes; a failed write does not discard a completed provider answer.

4.10 Organization, Workspace, and Private Identity

In the authorization model, Alice and Carol belong to Organization A and Bob to Organization B. Alice can write an eligible completed answer to a shared workspace and Carol can read it only with matching membership and `cache:shared` permission. Alice's private subject/application scope has no read path from Carol. Bob's similarly worded request resolves to Organization B's separate opaque identity; the denied cross-organization attempt is an authorization boundary rather than a low similarity score. The diagram is conceptual and does not expose raw credentials or Redis key material.

Automatic scope resolution comes from server-side directory membership, defaults, content eligibility, and provider profile. An explicit API request for a scope cannot grant missing permission.

Provider-profile compatibility is a second constraint: even two authorized members need the same compatible route identity for reuse. A response containing private context can be narrowed to Alice's private scope despite an originally eligible workspace request. Such classification is heuristic, so it is not a complete privacy guarantee.

4.11 Logical Redis Storage and Index

The Redis storage is best understood as a logical model, not a relational schema. A versioned namespace encodes policy, model, and provider compatibility. Within it, an opaque authorized scope tag and a digest of the stored question identify an entry hash. The hash contains question and answer text, the tenant tag, optional response-scope metadata, and, for RedisSearch semantic mode, a binary FLOAT32 embedding vector. The HNSW index uses the tenant TAG filter and COSINE distance to shortlist nearby entries. Approximate search can miss a suitable candidate; it does not assess the answer's applicability.

Redis membership sets and sorted sets record IDs, recency, sizes, organization expiry accounting, and quota totals. The entry hash has a finite TTL. A HIT touches LRU ordering, not TTL; expiry can occur without another lookup. Quota pressure can evict a young entry independently of its age. Redis itself needs a finite production memory setting. A process-local vector matrix is an alternative backend with different lookup and synchronization behavior, not an additional RedisSearch table. Redis never calls the LLM provider.

4.12 Local Deployment and Process State

Figure 4.5 depicts the actual local Compose configuration and the separate external-provider variant. The gateway image contains FastAPI, adapters, and application policy artifacts. The local `MockProvider` executes inside the gateway process. An OpenAI-compatible or Gemini profile sends HTTP traffic to the configured external provider, independently of Redis protocol traffic. Redis Stack has a named `redis-data` volume mounted at `/data`; the base Compose file declares no gateway model or configuration mount. The organization-demo overlay illustrated here adds a read-only configuration mount and a persistent model-cache volume. Models can load into each worker's own memory, while Redis entries, index, admission and quota state are shared.

Locks, memoization, and JSON singleflight are process-local. A second worker can see Redis entries but can still make a duplicate provider call and adds its own model memory. A managed multiworker deployment is a future scale-out design, not a measured capability of this local diagram. The measured mock and controlled-provider runs do not establish external-provider cost or latency.

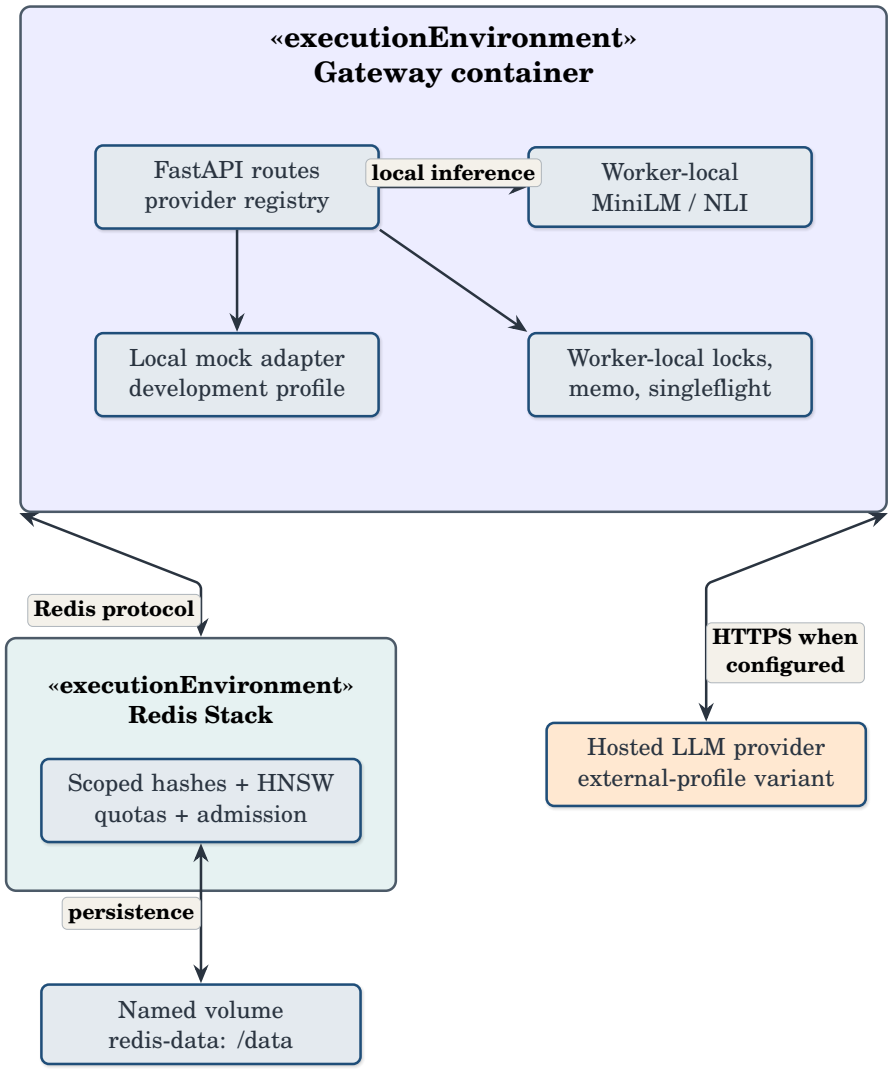


Figure 4.5: Compose containers, shared Redis state, and optional external HTTP.

4.13 Startup, Readiness, and Cache Residence

Startup begins with configuration validation and a Redis connection. Startup loads the organization directory, constructs the cache, model wrapper, registries, rate limiter, and metrics. Semantic MiniLM mode warms an embedding; configured answer-policy prewarm loads the verifier. RedisSearch semantic mode probes the vector dimension and creates or validates the HNSW schema. A configuration, Redis, model, or schema failure can prevent readiness. In exact mode the semantic model and index branches are not required.

The actual probe routes have distinct boundaries: `/live` gives a minimal process response, while `/ready` and `/health` share the dependency check in `app/main.py`. The Dockerfile's `HEALTHCHECK` calls `/health`, so its status includes Redis and configured component checks rather than liveness alone. In production these probe responses expose only the status, while `/admin/config` is authenticated and returns details. There is no separate, nonexistent liveness API.

The cache lifecycle separates processing from entry state. A completed response may be ineligible, explicitly no-store, a provider fallback, or a failed insertion; the generated answer can still be returned. A successful insert creates a hash and accounting metadata with a finite TTL. From the STORED state, lookup can update recency, TTL can expire the hash without a lookup, quota pressure can evict it by LRU, an authorized clear can remove it, or a same-key write can replace it. These transitions differ operationally: a touched entry may still expire, and a replacement begins a new residence period. A final candidate re-read catches a hash changed or removed while assessment was in progress.

Consider one completed technical answer stored at noon with a finite TTL. A HIT shortly afterward moves its entry to the recent end of the scope and organization LRU order, but its hash keeps the original expiry. If that TTL elapses, Redis can remove the hash even without another request; a later vector result must be validated against the missing hash and cannot revive it. If the scope fills before expiry, an insertion can evict the least recently used live entry instead. These are distinct removal mechanisms and neither checks whether the technical advice remains factually current.

A same-key completed write replaces the answer and starts a new stored generation. A shortlist produced before that replacement is not permission to serve the old answer: the final read compares the chosen question and answer with the indexed candidate. An authorized private-cache clear similarly changes membership before a later lookup. The user-facing response can still be a provider MISS if a write was refused or failed, with storage metadata explaining why the generated answer was not retained.

Chapter 5

Implementation

5.1 Gateway Request Path

The FastAPI routes [14] in `main.py` authenticate the supplied gateway key and resolve a principal through `OrganizationDirectory`. The directory selects a permitted scope and provider profile; the registry uses server-owned provider configuration. Rate admission precedes lookup. The request text is classified for no-store or private handling, and volatile requests, whose answers can drift as underlying facts change over time [5], bypass reuse. The non-streaming handler returns a structured JSON response containing outcome, provider, scope, candidate trace, and stage timings. The streaming handler emits provider events with completion handling. These routes implement a specific gateway contract, not every OpenAI API operation.

1: Validate request; authenticate principal and resolve provider registry
2: Apply rate limit; form prompt and resolve authorized scope, route, eligibility
3: Lookup private overlay where applicable; then scoped cache unless bypassed
4: **if** current exact answer or approved semantic candidate exists **then**
5: Recheck principal and return HIT
6: **end if**
7: Apply admitted miss/bypass count and request deadline
8: Call configured provider route; require normal completion
9: Classify response; recheck authorization and intended provider
10: Attempt eligible storage in workspace or private scope, subject to TTL/quota
11: Return MISS or BYPASS with provider outcome

Algorithm 1: Implemented request decision at the fixed checkpoint

5.2 Retrieval and Candidate Selection

Exact mode hashes the request text and reads the corresponding Redis hash. Semantic mode first tests the same exact key. Otherwise `Embedder` encodes the incoming question with pinned MiniLM through the sentence-transformers library [17] in the measured semantic profile. The RedisSearch backend on Redis Stack [15] stores a vector alongside the question, answer, scope, and entry fields and retrieves a scope-filtered top-k shortlist through its HNSW index. The alternative in-process backend keeps a matrix and rebuilds process-local snapshots under locks. In neither backend does a vector score establish that an answer is suitable.

With an answer-policy artifact, each shortlisted candidate passes bounded structural checks,

optional named-work and technical/applicability guards, and, if unresolved, an answer assertion NLI score. The assertion contains the stored question, stored answer, and incoming question. The policy records ACCEPT, VERIFY, or REJECT and a reason for each candidate. It may abstain because of a changed qualifier, long input, unavailable model, timeout, or incomplete evidence. Accepted candidates with conflicting answer texts cause a miss. The shortlist and verifier budget are finite; unexamined plausible candidates prevent an unqualified hit. A selected entry is read again and compared before serving, limiting replacement and expiry races.

5.3 Credentials and Automatic Scope Resolution

A gateway key and a provider key serve different purposes. The gateway key identifies the caller and its permissions. A server-configured provider profile identifies the upstream model and references its credential. The credential is resolved on the server when generation is required; it is not fetched from a user's consumer subscription. A HIT avoids that generation call. A MISS consumes the configured provider account's resources, subject to the provider's billing arrangement. VeriGate's request admission counter is separate from token billing.

For normal application traffic, the client omits the optional scope override. The directory selects the application's authorized default workspace. A private-content classification can restrict the request to the employee/application scope, and a secret classification prevents cache use and storage. Response eligibility is checked again before writing because a seemingly general question may receive a private answer. If no shared default is authorized, the request remains private. These decisions are made by the gateway, not by asking the employee to label every prompt.

Shared reuse also requires compatible provider profiles and routes. Two employees in the same organization are not automatically interchangeable across all workspaces or model configurations. These boundaries prevent a convenient shared cache from becoming an unrestricted organization-wide response store.

5.4 Organization Example

The following is an illustrative flow, not a measurement. Employee A asks how to inspect a deployment in a shared technical workspace. A normal provider completion is stored only if the request, response, and authorization permit workspace sharing. Authorized employee C asks a paraphrase in the same workspace. The gateway retrieves A's stored question and answer, checks that C's requested action and environment remain compatible, and may return a HIT after a final entry check. If C asks to *apply* a change while A's answer only explains how to *preview* a Terraform plan, an applicable guard should decline that candidate and use provider fallback. The measured development and lockbox behavior of such action changes is reported separately in Chapter 6; the generality of this example is not established by the illustration.

5.5 Providers, Storage, and Concurrency

The provider registry selects an intended route and supports configured failover and a circuit breaker [11]. The development mock is a deliberate provider choice, not a permanent tail of every live chain. A JSON response is cached only after a nonempty answer and a normal terminal completion; overly long, timed-out, or failed upstream responses produce errors. Streaming has its own first-token and idle deadlines and completion checks. Fallback output from a different intended provider is not written into the intended route's cache.

Redis holds entries, TTL and LRU metadata, admission counts, and, with RedisSearch, the shared vector index. Configured entry quotas restrict a scope; Redis's own memory limit is a separate operational capacity. Singleflight coalesces compatible identical JSON misses within one process; the streaming path does not provide the same coalescing. The in-process vector matrix, model weights, memoized verifier results, and locks are local to a worker. The namespace hashes settings and artifact versions so incompatible cache policies do not silently share old entries. Namespace retirement and TTL are not a complete immediate purge mechanism.

5.6 Configuration Profiles and Observability

Table 5.1: Representative configurations; flags are independent choices, not one combined production mode.

Profile	Matching and storage	Provider and verifier
General defaults	Exact; in-process vector option unused; finite TTL and entry quota	Explicit mock development provider; no answer artifact
Semantic demo	Semantic; MiniLM and RedisSearch when configured	Pinned answer policy and NLI, prewarm recommended; mock labels remain visible
Real-provider setup	Exact by default; scoped Redis	Server-owned OpenAI-compatible or Gemini profile; live credentials required
Experimental flags	Technical and applicability guards off by default	Optional task verifier requires a validated local artifact

Readiness checks include Redis, index, and configured model preparation; optional prewarming moves model load to startup. Prometheus counters and histograms [13] distinguish hit, miss, bypass, candidate decisions, and stage timing. Logs and candidate traces support debugging without treating an NLI score as a factual truth label.

5.7 Experimental Task Verifier

The repository includes an executable task-specific reuse-verifier pipeline that forms an input from stored question, stored answer, and incoming question, separates training and calibration, and loads a manifest with artifact hashes and a threshold. The saved 75-row, five-family source is authored development material. At this checkpoint no trained, independently validated artifact has passed a promotion gate for the selected gateway profile. The pipeline is implemented experimental work; it is not a measured improvement or a replacement for the pinned NLI results in Chapter 6.

5.8 Source Map and Runtime Ownership

Table 5.2 identifies the principal implementation modules. The gateway is intentionally split between HTTP orchestration, scope policy, candidate selection, vector storage, and provider adapters. This avoids treating a successful route handler as proof that the retrieval or verification policy is correct. Each component has a narrower contract that can be tested independently.

Table 5.2: Code-grounded module map at the recorded source checkpoint.

Module	Responsibility and main output
`main.py`, `schemas.py`, `limits.py`	HTTP lifecycle; single-turn request validation; bounded body/concurrency; JSON or stream response.
`organization.py`, `identity.py`	Credential-to-principal lookup, permitted scope, provider profile, opaque identity.
`semantic_cache.py`	Exact shortcut, vector lookup, top-k decisions, final entry check, and scoped store.
`redisearch_store.py`, `storage.py`	Redis hash/index access, atomic membership, TTL, LRU, byte and entry quotas.
`embeddings.py`	Lazy pinned MiniLM encoder or deterministic lexical hash backend.
`answer_policy.py`, `verifier.py`	Candidate structural decisions and pinned NLI answer-assertion score.
`applicability.py`, `technical_identity.py`	Optional bounded action, environment, and identity checks.
`providers/registry.py`, adapters	Intended route, breaker, first-token/idle deadlines, explicitly configured failover.
`singleflight.py`, `metrics.py`	Local duplicate JSON miss coalescing and operational counters/histograms.
`static/dashboard.html`	Optional operator/demo UI that constructs requests and displays actual outcomes.

5.9 Startup, Readiness, and Local Versus Network Work

The settings model reads environment values and an optional local environment file. On lifespan startup, the service creates text and binary Redis clients and checks connectivity. An organization directory can load hashed gateway-key digests and provider profiles. The gateway creates one embedder and semantic-cache coordinator for that process, builds profile-specific provider registries, and initializes rate limiting and metrics. If semantic mode uses MiniLM, a warmup encode can force the model to load. When the answer policy requires prewarming, failure to initialize the verifier prevents readiness. RedisSearch semantic mode probes the vector dimension, creates or validates the HNSW index, and fails if the existing vector schema is incompatible.

MiniLM and NLI inference execute locally in the gateway process and do not make a paid provider call. They can consume substantial CPU time and resident memory. Redis operations use a network protocol to the selected Redis server, even when it runs on the same machine. Only a miss or bypass invokes the configured upstream LLM adapter. With the development mock or controlled provider, that provider is local and deterministic. With an OpenAI-compatible or Gemini profile, the adapter makes an external HTTP call using a server-owned credential. The gateway cannot infer that such an adapter works with every endpoint merely from its construction; a bounded real-provider smoke remains a separate evidence gate.

Liveness is a minimal process-level status. Readiness includes selected dependencies and model/index state, while production validation also rejects unsafe default secrets and missing shared Redis configuration. Operational status can change after startup, so a startup check does not replace ongoing monitoring. Prewarming mainly moves a cold model load before readiness. It does not eliminate the computational cost of each new assertion or embedding.

5.10 Request Schema and Authentication

The supported request is single-turn. It supplies either a prompt of at most 16,000 characters or one user message with the same content limit. Extra unsupported fields are forbidden. Optional fields cover cache bypass, a private/workspace/none scope request, and a workspace identifier within its length bound. The body middleware also limits buffered bytes and active requests. These validation decisions make failure visible; silently accepting a full conversation history or unsupported generation settings would misrepresent the service's contract.

An application sends its gateway key in the `x-api-key` header. The directory compares its digest and resolves a principal rather than using the key text as a workspace identifier. That principal's organization, subject, application, memberships, permission set, and provider profile determine the allowed cache scope. The provider profile's secret comes from server configuration. An operator may rotate or revoke a gateway credential independently of the upstream provider credential. The handler rechecks the principal before returning a slow HIT and before provider admission. After generation, the recorded checkpoint checks authorization before cache storage: a revoked principal suppresses the write, but the completed JSON answer can still be returned. This is not a guarantee that revocation cancels an in-flight response.

As an illustrative API input, a trusted application can send

```
POST /v1/chat
x-api-key: example-gateway-key
{"prompt":"How can I inspect the staging deployment?"}
```

The omitted scope fields mean automatic server-side selection; the example credential is fictitious. A developer-only override may request `cache_scope=private`. An unauthorized explicit workspace request receives an authorization error rather than creating access through the API field. A normal dashboard request follows the automatic path. The inspected dashboard also exposes explicit controls in a separate developer mode and labels mock placeholders distinctly from cached or generated answers. Those UI observations refer to the committed dashboard at checkpoint `1cdb80a`; later UI changes require separate reconciliation.

5.11 Embedding and Scoped Retrieval

MiniLM converts each question into a 384-dimensional normalized FLOAT32 vector. An embedding is a numeric representation intended to put related text near one another. Cosine similarity is the dot product of normalized vectors; it ranks how close the question representations are. A high score is compatible with a changed action or environment and is never, by itself, an answer-approval rule. The test-only hash backend instead counts normalized tokens into a deterministic vector; it is fast and dependency-light but lexical rather than a substitute for the measured semantic model.

In semantic RedisSearch mode, the index has a TAG field for the opaque scope and an HNSW vector field for cosine distance. The query requests a bounded top-k set, with limited overfetch to compensate when a returned hash has expired. Each search result is matched against its current Redis hash for scope, question, and answer identity before entering the candidate list. A process-local matrix is the alternative backend. It still reads Redis as the authoritative store but keeps a local vector snapshot and locks around rebuilds. A NumPy dot-product scan of that matrix is linear in entry count, not a proven sub-linear index. The shared RedisSearch index is more suitable for multiple workers, but its approximate retrieval recall and large-scale capacity have not been established for this project.

The exact shortcut computes a question digest and reads its scope-local Redis hash before embedding. This is an optimization for identical text; it is not the main mechanism for paraphrase reuse.

It rereads after updating recency so an expired or replaced entry is not served from an earlier local view. In semantic mode, candidates are evaluated in rank order. A lower-ranked candidate can win if higher-ranked candidates are declined, subject to the limit, deadline, and conflict rule.

5.12 Answer Suitability and Selection

For each non-exact candidate the answer policy receives the incoming question, stored question, stored answer, and retrieval similarity. Its first checks reject malformed or too-distant entries and changed numbers, polarity, named works, and answer type. Optional technical and applicability checks examine bounded syntax such as requested action, HTTP method, target environment, version, permission, or named workspace when those facets are supported and grounded in the recorded answer. An unsupported or ambiguous phrasing can abstain. These checks deliberately trade some valid reuse for a more explicit reason to decline a changed task; they do not understand arbitrary technical requests.

An unresolved candidate is sent to a pinned local NLI cross-encoder as a pair of assertions: the stored answer is asserted for the stored question and then for the incoming question. The model's entailment probability is a learned signal about compatibility of those statements. It does not verify that the stored answer was factually true, current, complete, or authorized for a new audience. Paired inputs are tokenized without silent truncation; if they exceed the model window, the policy returns VERIFY. Unavailable inference also produces VERIFY. The policy artifact supplies distinct accept and reject thresholds, leaving an uncertainty interval. A candidate action of VERIFY means abstain and use the configured provider path if no other candidate is accepted; it does not call a human reviewer.

The selector records a trace of each considered rank, similarity, action, reason, score status, and elapsed time. A configured verification limit and millisecond budget cap work on the critical path. If plausible lower candidates remain unassessed, or accepted candidates disagree on answer text, the entire request becomes a MISS. After choosing an accepted candidate, the coordinator reads the original hash again and compares the question and answer. A disappeared or changed entry is not served. This last check addresses a practical race between vector search, model inference, expiry, and concurrent writes.

5.13 Provider Completion, Retention, and Errors

The registry resolves an intended model route from the principal's provider profile. Optional complexity routing can choose a cheaper or stronger model, but that route is an engineering heuristic and its answer quality has not been validated. Circuit-breaker tickets track transient failures; the registry can try only providers that were explicitly configured. It never invents a mock fallback for a real provider failure. First-token, idle-stream, provider, lookup, and overall request deadlines protect different stages. An exceeded overall deadline can prevent provider dispatch entirely.

JSON generation assembles content events and requires one normal completion with a stop reason. Empty, oversized, interrupted, or non-normal completions return errors and are not cached. Streaming emits content and metadata events and only emits a final completion marker after success. An interrupted stream must not become a reusable partial answer. A completed response is classified for storage. A request or response marked no-store is not written. A private-only response from a workspace request uses a private scope. A successful fallback through a different provider is not stored under the intended provider's compatibility identity. A storage failure is reported while retaining the already completed generated answer.

The cache TTL, scope LRU, and organization count/byte quotas constrain residence and memory usage in different ways. TTL is time since insertion; LRU chooses a victim under pressure. Occupancy reports are useful for operations but do not measure answer quality. Rate limiting applies before lookup, while the daily admitted-miss count is reached only after a miss or bypass. Because it counts requests rather than billed tokens, it cannot enforce a monetary spending cap.

5.14 Four Request Walkthroughs

First request (illustrative). Alice's integrated application sends an eligible technical question with no scope override. Her principal resolves to Organization A's authorized workspace and its provider profile. No current exact or semantic candidate is approved, so the request is admitted for a provider call. If the configured provider completes normally and its answer is shareable, the gateway stores question, answer, vector, metadata, and TTL under the compatible workspace identity. The JSON response reports MISS and the provider name. If a cache write fails, Alice still receives the complete generated answer with a failure storage status.

Colleague's paraphrase (illustrative). Carol has membership in the same workspace and compatible profile. Her differently worded request misses the internal exact shortcut. MiniLM produces a vector, RedisSearch returns Alice's stored entry, and the policy checks the particular answer against Carol's task. If it accepts and the final hash is unchanged, Carol receives a HIT without provider generation. The saved isolated release probe recorded such a seeded decorator-answer hit, but that selected example does not establish general answer accuracy.

Changed task (illustrative). Alice's stored answer explains how to preview a Terraform plan for staging. Carol asks to apply a production change. The questions can be near each other in vector space, yet the action and environment differ. An enabled applicability guard can reject the candidate before model scoring; if a guard is off or cannot parse the request, the NLI path may still accept or abstain. A declined candidate yields a provider miss, which is desirable when reuse would be unsuitable. The controlled evaluations in Chapter 6 document both prevented and missed cases.

Private or no-store request (illustrative). Alice asks about account-specific information. A private explicit scope or the server-side eligibility decision routes storage into Alice's subject/application namespace. Carol's workspace lookup cannot read it, and Organization B has an entirely different scope identity. A no-store decision bypasses cache reuse and suppresses storage after generation. This is a bounded sharing policy, not a guarantee that a heuristic identifies every sensitive string.

Chapter 6

Results and Evaluation

6.1 Evidence Types and Metrics

The results are separated by what they actually observe. Functional tests check code behavior; pair-proxy studies label relations between questions; controlled answer studies stipulate whether a particular recorded answer fits a later task; HTTP studies include retrieval, selection, and provider fallback. Valid reuse divides suitable answers served by labelled valid opportunities. Incorrect acceptance divides invalid requests served from cache by labelled invalid requests; ambiguous approvals have their own denominator. A VERIFY count denotes abstention from a candidate, whereas provider fallbacks include all requests without a hit. Latency percentiles are reported with their path and measurement boundary.

6.2 Functional and Integration Verification

The generated check table (source `evaluation/results/checks.json`) now records 339 passed, 13 skipped, zero failed, and zero errors in 352 collected tests, regenerated from the default fakeredis suite rather than carried over; the 13 skips are opt-in integration tests (disposable Redis Stack, the pinned-NLI contract, the Playwright browser regression) that run in their own jobs instead of being silently dropped. This default suite is not a correctness measurement. On the follow-up branch's exact commit the ordinary tests job (including the generated-table diff gate), the lint and type checks, the browser regression, the dependency audit, and the Docker build passed in GitHub Actions; real-MiniLM/NLI and RediSearch run in the Redis-stack job where reachable. Saved organization HTTP observations with mock profiles show authorized cross-employee workspace reuse and private or other-organization isolation for their selected cases, and gateway tests cover auth, completion, streaming, deadlines, scope, and candidate identity. None of these counts measures answer correctness in general.

6.3 Historical Query-Pair and Proxy Experiments

The initial candidate-pair smoke fixture and later QQP, PAWS, and STS-B studies used question-pair or sentence-similarity labels. Their candidate was often stipulated rather than retrieved, and no actual cached answer was judged for the new request. The filling-cache replay added TTL, LRU, and tenant partitions but marked cross-pair outcomes unknown when labels did not cover the retrieved candidate. Five overlapping seed splits are descriptive, not five independent production samples. PAWS transfer of the QQP-trained scorer was weak, and the strict pair-proxy

risk selector chose reject-all at tested targets. The archived tables are retained in Appendix A; they do not establish a deployed answer-error bound.

6.4 Judge Validation and Public-Query Pilots

The initial judge experiment evaluated 15 authored binary controls plus one ambiguous case. FLAN-T5-small answered NO for every binary control and scored 7/15, with zero recall on eight valid cases. Pinned DeBERTa NLI scored 13/15, accepting six of eight valid cases and rejecting all seven invalid controls, but gave a high acceptance score to the separate ambiguous case. This was model selection evidence, not a general answer-quality certification. On a separate unlabelled queue of 200 diagnostic and 150 random pairs, the judges disagreed on 95 and 73 rows respectively. Those 168 disagreements cannot be counted as factual errors. The saved judge-validation record preserves that distinction.

A public NQ-Open dev pilot used 60 stored questions, 50 distinct incoming questions and ten exact repeats, excluding recorded development overlap. It used real MiniLM vectors and nearest-neighbour selection, but no independently validated reuse labels. In version 1, exact, static and adaptive policies accepted the ten repeats; answer-aware reuse accepted only five because floating-point cosine values slightly above one were rejected. A separately versioned numerical-tolerance fix accepted all ten repeats while retaining rejection of all 50 distinct random neighbours. Of those 50, 41 were below the retrieval threshold: the workload mostly tested distant candidates, not difficult paraphrases. Version 3 warm local lookup p95 was 58.46 ms; it excluded Redis, HTTP, generation and model loading. The pilot record retains the failed original run and the follow-up rather than replacing old evidence.

These stages explain why later evaluations moved to answer-bearing technical tasks, competing candidates and real HTTP paths. Public input provenance alone does not make the labels independent or make an offline timing equivalent to gateway latency.

6.5 Controlled Answer-Aware Evaluations

The authored `answer_aware_20261029_v6` set contains 60 binary cases from 20 fact families and two ambiguous cases. With a fixed designated candidate, lexical hash embeddings, and warm decision timing, the answer-aware policy accepted 16/20 valid paraphrases, 0/40 changed or conflicting cases, and 0/2 ambiguous cases. Its 48.91/62.54 ms p50/p95 excludes embedding, retrieval, HTTP, and generation. The static and adaptive query-only baselines answer different questions; their errors on this development set cannot be read as a deployed comparison.

The frozen 32-case, eight-family technical applicability set supplied one fixed candidate per request, real MiniLM similarity, and pinned NLI. The baseline served 9/16 valid cases, accepted 1/8 invalid and 5/8 ambiguous cases; the optional applicability guard served the same 9/16 valid and accepted 0/8 invalid and 0/8 ambiguous. Its VERIFY count rose from 7/32 to 14/32. These authored task contracts support a narrow action/environment check, not broad factual accuracy. A separate 30-case, ten-family bidirectional assertion test served 4/10 valid with the current forward check and 3/10 with the reverse addition, with 3/10 ambiguous approvals in both. The reverse addition was not promoted.

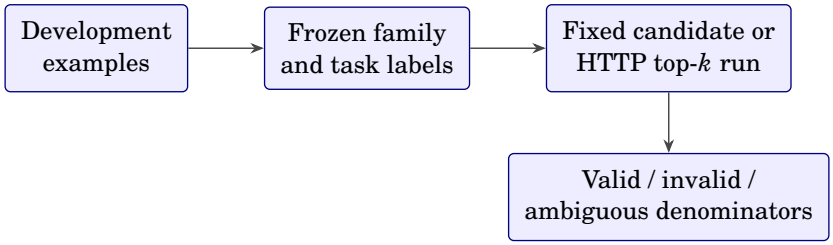


Figure 6.1: Evaluation provenance. Development examples are diagnosed separately; frozen authored labels remain a limited substitute for independent real-user review.

6.6 Real Retrieval and HTTP Path

The 25-probe applicability challenge used five authored families: 15 valid, five invalid, and five ambiguous requests, with three seeded answers per probe. It ran through HTTP, RedisSearch top-$k = 5$, MiniLM, pinned NLI, and a controlled provider. Suitable candidates appeared in every valid shortlist, yet both baseline and guarded variants served only 4/15 valid answers; 11 valid misses came after retrieval. Both had 0/5 invalid accepts and 0/5 ambiguous approvals, 21/25 misses after candidate assessment, and 21 controlled-provider calls. Candidate traces yield 10/25 request-level VERIFY abstentions; the historical summary field called `verification_abstentions` actually counted all 21 assessed misses, including rejections. That field is preserved in source data and clarified here.

The later fresh `technical_combined_fresh_20261101_v1` workload froze 25 new authored probes before the run. It compared the pinned NLI baseline with both technical and applicability guards enabled through HTTP, RedisSearch, MiniLM, and a controlled provider. On the saved forward-verified run, baseline valid reuse was 6/15 and combined-guard reuse 8/15; invalid acceptance was 0/5 in both, ambiguous approvals 1/5 versus 0/5, and retrieval failures 0/15 in both. VERIFY request abstentions were 11/25 versus 9/25; reject-only misses were 7/25 versus 8/25; provider fallbacks were 18/25 versus 17/25. The predeclared promotion gate demanded at least three additional valid hits, so the observed two did not pass. The reverse-order run repeated the same outcome counts. These are two orderings of the same 25 authored tasks, not 50 independent cases.

Table 6.1: Distinct HTTP workloads with controlled provider. Counts are per frozen workload; rows must not be pooled.

Run and variant	Valid reuse	Invalid accepts	Ambiguous approvals	Retrieval failures	Provider calls
Earlier applicability OFF	4/15	0/5	0/5	0/15	21
Earlier applicability ON	4/15	0/5	0/5	0/15	21
Fresh baseline	6/15	0/5	1/5	0/15	18
Fresh combined guards	8/15	0/5	0/5	0/15	17

6.7 Latency and Resource Measurements

The earlier 25-probe run measured subsequent HTTP completion at 341.792/1458.667 ms p50/p95 with the guard off and 155.311/279.052 ms on. It was one sequential OFF-then-ON run on a Windows machine; warm state and scheduling may differ, so this is not a repeatable speedup estimate. The fresh forward-verified run measured warm HTTP p50/p95 of 340.254/519.109 ms baseline and 154.414/329.077 ms combined, also with a controlled provider. Its reverse ordering

measured 410.687/597.544 ms and 180.123/307.189 ms respectively. The runs show synchronous verifier cost can dominate local mock generation, but do not measure paid-provider savings.

The saved 30-trial prewarm benchmark used Redis Stack and delayed mock generation on one CI runner. A first NLI hit took 1334.897 ms with lazy load and 59.196 ms after prewarm; warm same-query hit p50/p95 was about 12.5/19 ms in both modes. Prewarm moved model work into startup rather than making inference free. A separate one-process memory snapshot rose by 680.9 MB from import to pinned NLI prewarm; models are process-local, so worker count matters. The saved Redis shortlist microbenchmark measured 80 local rounds of candidate-hash validation only: sequential median 3.357 ms and pipelined median 0.896 ms. It is not full HTTP latency or a scaling result.

6.8 When Caching Can Reduce Cost

Cost effectiveness is the design objective, not a measured monetary result in these experiments. Let N be the number of requests in a fixed billing period, h the fraction served from cache, c_p the average avoided provider charge per hit, c_g the average incremental gateway cost per request, and C_f the fixed incremental hosting cost for the period. Under the simplifying assumption of a comparable provider workload, the net saving is

$$S = Nhc_p - Nc_g - C_f. \tag{6.1}$$

The break-even condition is $hc_p > c_g + C_f/N$. Gateway cost includes embedding, verification, Redis, and hosting rather than treating HIT as zero-cost. In practice token counts and provider charges vary by request, so a billing study must sum the avoided charges of the particular requests that hit. The equation is an analytical model, not a fitted result.

For illustration only, suppose 100 eligible requests would each cause one provider call without caching and 30 are correctly reused. The gateway would make 70 generation calls, assuming no retries or additional provider-based judging. That is 30 avoided calls, not automatically a 30% reduction in the total bill. Organizational sharing increases the pool of potentially repeated tasks within an authorized workspace; it does not pool private answers or cross organization boundaries.

Latency has a similar condition. Let L_d be mean completion latency for a comparable direct-provider workload, L_h mean gateway HIT latency, and L_m mean gateway MISS latency, including generation. Ignoring bypasses for this simplified model,

$$L_{\text{gateway}} = hL_h + (1-h)L_m. \tag{6.2}$$

The gateway improves mean completion time only when this quantity is below L_d. This weighted-mean calculation must not be applied to p95 values. Slow verification can erase the benefit of avoided calls, especially when valid reuse is low. A real-provider study should therefore measure charges and latency for the same request mix, model, output limits, and operating conditions.

6.9 Follow-up Audit Fixes and Controlled HTTP Demonstration

A later audit pass fixed three verified issues without changing acceptance decisions, routing, or cache identity. The exact-revision CI failure was a stale generated artifact: the committed test-count summary still read 303/7 while the suite had grown, so the diff gate that regenerates and compares it failed; regenerating it (339/13) restored the gate. The opt-in Playwright dashboard regression and the pinned-NLI contract test were never enabled in CI and so skipped while the pipeline still reported success; dedicated jobs now set the flags, install a pinned browser outside the locked runtime set, and fail if a required test silently skips.

The third and most visible fix is the answer-source label. The dashboard previously fell through to "generated answer" for any provider that was neither the cache nor a mock, so a response from the development controlled-answer provider, a fixed labelled fixture, appeared as a genuinely generated answer. The label now comes strictly from the returned provider metadata: "controlled evaluation fixture" for the controlled provider, an explicit mock placeholder, "cached answer" for a cache delivery (without asserting the original provenance), a named real provider, or "unavailable" when none is reported. The earlier heuristic that inferred provenance from known answer text was removed, and an opt-in browser regression asserts that a controlled fixture is never shown as generated. Figure 6.2 shows the corrected panel for one such request: alongside the returned answer it reports the observed request journey, the measured gateway stages, the provider `controlled-eval-example-eval`, the `workspace` lookup scope, the `stored` write outcome, and "NLI model evidence: not invoked" because the exact shortcut returns before the verifier runs. Reading the source directly from this metadata, rather than from the answer text, is what keeps a fixture from being presented as a model-generated answer.

To demonstrate the honest reuse path end to end, the controlled-answer provider was also enabled on the organization path (development only, secret-less, rejected in production). A scripted HTTP run against a disposable gateway then reproduced the shared-reuse, isolation, workspace-denial, provider-failure-without-write, and TTL-expiry behaviours of Tables 6.3 and 6.4 with real labelled answers instead of a mock echo, exercising the exact shortcut rather than the MiniLM/NLI semantic path (which needs model downloads and runs in CI). This is a controlled demonstration, not accuracy, cost, or production-readiness evidence.

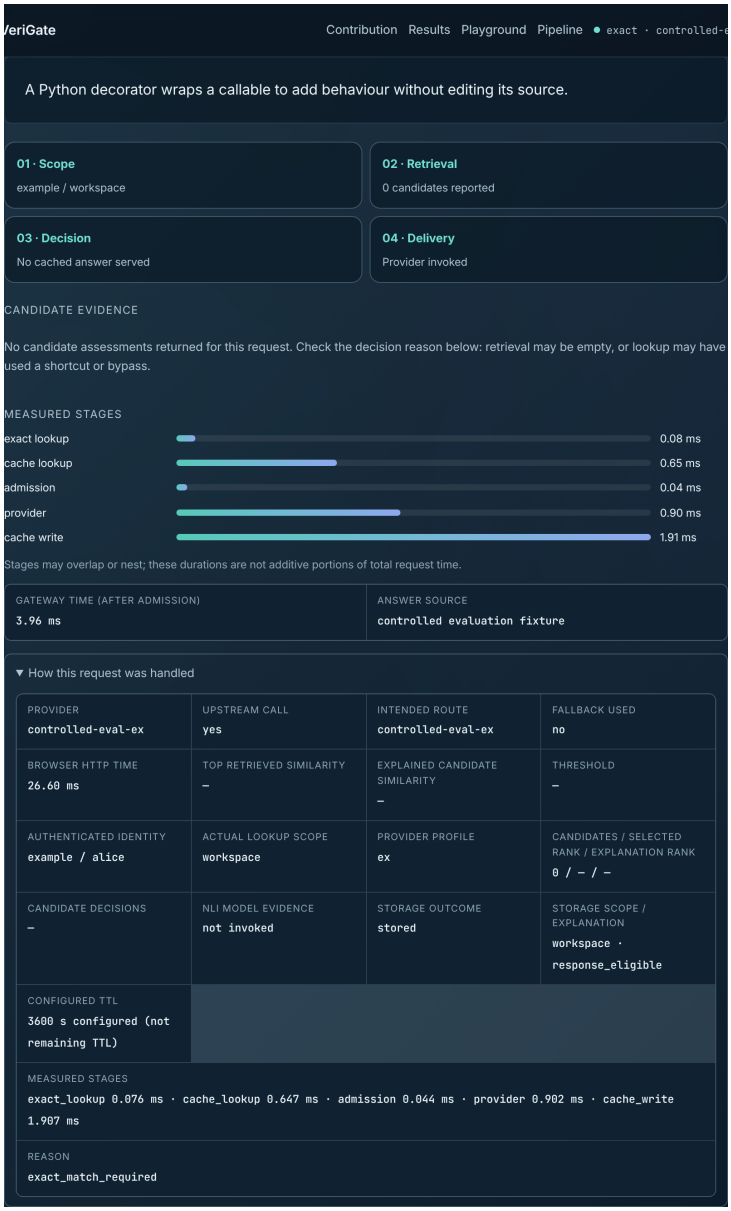


Figure 6.2: Dashboard result panel for a controlled-evaluation fixture request, captured after the answer-source correction. The always-visible *answer source* reads “controlled evaluation fixture” (never “generated”); the expanded “How this request was handled” panel shows the actual provider, the workspace lookup scope, the `stored` write outcome, and that the verifier was not invoked on the exact-shortcut path. A labelled development fixture shown to illustrate the corrected provenance labelling and the evidence the dashboard surfaces, not generated-answer, accuracy, or cost evidence.

6.10 Limitations and Threats to Validity

All technical suitability labels above are authored stipulations, not independent adjudication of real employee traffic. They do not test truth of the cached answer, prevalence of changed actions, privacy detection recall, or long multi-step tasks. The earlier 4/15 and fresh 8/15 results differ in task families and policy variants; they are not a before/after improvement on one dataset. A model score or judge disagreement is not an answer-error rate. Zero observed invalid accepts among five examples cannot support a low general error bound. The task-specific verifier has an

executable pipeline but no promoted validated artifact. Real-provider compatibility, billed cost, broad concurrency, and scale remain unverified at this checkpoint. Revocation during generation prevents storage at the recorded revision but does not necessarily suppress the completed response; the semantic identity read and recency update are also separate operations.

6.11 Testing Strategy and Execution Layers

Testing was organized around claims rather than a single pass count. A unit test isolates a rule such as scope resolution, request schema rejection, numeric mismatch, answer-policy action, freshness bypass, TTL accounting, or overlong verifier input. It can detect a deterministic branch error but does not prove the assembled service behaves the same way. Integration tests add Redis or RedisSearch and exercise candidate identity, scope filtering, quota eviction, and shared state. HTTP tests use FastAPI's client or a real local server to test authentication, request outcomes, provider completion, metadata, and storage. Failure and concurrency tests inject slow or broken components so the system reaches paths that a normal demonstration would rarely encounter.

The repository CI separates ordinary tests, Redis-backed tests, lint/type checks, a dependency audit, a Docker build, and selected live-path benchmarks. The generated `checks.json` summary now records 339 passed and 13 skipped from the default fakeredis suite on the follow-up branch, regenerated from the actual run rather than carried over. Earlier revisions committed a stale 303/7 count that no longer matched the grown suite, which made the exact-revision diff gate fail; regenerating the summary restores that gate. The 13 skips are opt-in integration tests: disposable Redis Stack cases, the pinned-NLI transformation contract, and the Playwright browser regression. A follow-up CI change now runs the contract and browser tests in dedicated jobs with their flags set and fails the job if a required test silently skips, so a present-but-skipped test can no longer be mistaken for a passing one. The default suite still does not measure answer correctness, and real-model and real-provider behaviour remain CI- or deployment-only.

Table 6.2: Testing layers and what their evidence can establish.

Layer	Representative assertions inspected	Setup and evidence	Boundary
Unit	Input constraints, guard decisions, freshness and verifier length	Gateway reliability, technical-constraint and NLI-bounds tests	Rule behavior only
Redis	Scope filter, mutation, expiry, LRU and quota	Top-k policy and scoped-capacity tests	Selected fixtures
HTTP/API	Auth, HIT/MISS/BYPASS, storage, metadata	Gateway/organization tests and isolated release probe	Local mock profile
Failure	Deadline, incomplete stream, write failure, coalescing	Reliability, deadline, completion and singleflight tests	No broad traffic load
UI/browser	Ordinary payload, developer override, answer-source labels	Dashboard tests + opt-in Playwright regression executed in CI	Controlled fixtures labelled, not generated
Semantic quality	Labelled fixed candidate and live top-k	Frozen authored workloads and saved rows	Not production truth
Performance	Stage, HTTP, prewarm, RSS, Redis read	Saved manifests and timing arrays	Local hardware and mock provider

6.12 Representative Functional and Failure Cases

Tables 6.3 and 6.4 list inspected assertions and saved HTTP observations rather than assigning success to every desirable behavior. An expected MISS prevents unsafe reuse when the task changes; an undesired MISS loses a valid opportunity. An error response is correct when admission or provider completion fails. A selected fixture is evidence for its mechanism, not a frequency estimate. Test assertions and saved probes are marked separately.

6.13 Semantic Evaluation Protocol

The answer-reuse unit is a triple: stored question, its stored answer, and an incoming probe. A valid label means the particular answer is stipulated to satisfy the incoming task. An invalid label means it is stipulated unsuitable, for example a plan instruction for an apply action. Ambiguous means the new request lacks enough scope to approve reuse. These authored labels are independent of model output but are not independently human-reviewed production judgments. Public NQ questions and references appear in the song stress set; authored templates and swaps create request variations. Public QQP/PAWS/STS-B labels are pair relations and cannot be converted into answer-reuse truth.

Fixed-candidate tests force the designated stored answer into the decision rule. They reveal whether checks and NLI approve that candidate, but cannot measure whether a real index would retrieve it or whether a competing answer would win. The live top-k HTTP challenges seed multiple answers into isolated Redis scopes, make actual HTTP calls, and retain candidate traces. Each valid candidate's presence in the shortlist is counted separately from its acceptance. This distinction matters: in the earlier 25-probe run all 15 suitable candidates were retrieved, while 11 valid requests missed because NLI declined their suitable answer. The source diagnosis records five VERIFY and six REJECT actions for those 11 suitable candidates. There was no retrieval failure on that selected workload.

Development examples were inspected to diagnose failures; later task families were frozen before evaluating variants. The applicability fixed-candidate set had 32 tasks from eight families. The bidirectional lockbox had 30 tasks from ten families with task-contract labels; its reverse check was an experiment, not a deployed method. The fresh combined-guard HTTP set had 25 tasks from five new families and a predeclared gate: at least three more valid HITs out of 15, no increase in five invalid or five ambiguous approvals, a bounded warm-latency criterion on balanced runs, and preservation of isolation regressions. Its forward and reverse orderings used the same underlying tasks. Because the combined guards added only two valid HITs, the gate failed even though the observed counts looked better. Retuning on inspected cases would turn a held-out comparison into development data.

6.14 The 1,000-Request Stress Study

The saved `nq_song_stress_20261030_v3` workload contains 200 song-writer families from NQ-Open train. Each family contributes one original question, three authored paraphrase forms, and one authored song-title swap. One seed shuffled the 1,000 unique request texts. Each policy used a fresh evaluation scope, Redis Stack, pinned MiniLM, and a fixed-answer controlled provider after a genuine miss; the answer-aware variant also used pinned NLI. The exact, static-0.72, and answer-aware configurations ran sequentially at sufficient and deployment capacity. The saved workload, freeze, manifests, request rows, and analysis preserve input and policy hashes. No paid calls were made.

Table 6.3: Representative test cases, part 1.

ID	Scenario and setup	Expected behavior	Observed result	Evidence and status
T01	Missing gateway credential; local POST	Reject before body work	401 in assertion	Gateway tests; archived suite
T02	Carol, authorized workspace; seeded answer	Share eligible answer	Semantic HIT, rank 1, no provider	Isolated mock release probe
T03	Organization B asks same task	Separate scope	MISS in other scope	Same selected probe
T04	Carol requests Alice private content	No cross-subject reuse	Private requests remained separate	Same probe; no general leak bound
T05	Ordinary UI request, no scope override	Server chooses scope	Inspected payload omits override	Committed in 1cdb80a; browser test source inspected
T06	Identical prompt in one scope	Exact shortcut HIT	First MISS, next HIT	Gateway test assertion
T07	Valid paraphrase to seeded answer	Answer-aware HIT	HIT with NLI score 0.9208	Selected release probe
T08	Changed task constraints	Decline unsuitable entry	Provider fallback in probe	Mock output is not answer truth
T09	Ambiguous technical request	Do not assume narrow answer applies	One baseline approval in five fresh ambiguous cases	Fresh controlled run; known negative

Table 6.4: Representative test cases, part 2.

ID	Scenario and setup	Expected behavior	Observed result	Evidence and status
T10	Empty scoped cache	Provider MISS	MISS in gateway assertion	Gateway test; archived suite
T11	596-token assertion, 512-token window	VERIFY, no truncation	VERIFY on frozen long case	Timed bidirectional lockbox
T12	Verifier unavailable	Abstain and use provider	Conservative action in verifier tests	Recovery needs final review
T13	Exact entry replaced during touch	Do not serve old answer	MISS, changed-entry reason	Top-k policy injected mutation
T14	Provider omits terminal completion	Error; no cache entry	502 JSON or stream error; retry MISS	Provider completion tests
T15	Completed answer, cache write raises	Keep answer, report failed write	200 JSON with mock answer	Reliability injected failure
T16	Scope/org quota pressure	Evict LRU within limits	Victim removed, occupancy bounded	Scoped-capacity assertions
T17	Duplicate concurrent JSON miss	Coalesce in one process	One upstream call in saved benchmark	Cross-worker duplication persists
T18	Stream interruption/cancellation	No partial stored answer	Interrupted stream retry MISS; early disconnect not fully established	Reliability test and review ledger F14

At sufficient capacity, exact mode made 1,000 provider calls because no question text repeated. Static semantic mode made 198 provider calls, recorded 712 reference-matching hits and 90 suspected wrong hits. Answer-aware mode made 264 provider calls, with 682 reference-matching hits and 54 suspected wrong hits. Those suspected wrong hits are short-reference mismatches requiring source review, not adjudicated answer errors; NQ references may omit aliases or co-writers. The answer-aware policy also missed 58 opportunities relative to its controlled reference criteria. Compared with static reuse, verification removed 36 suspected mismatches while losing 30 reference-matching hits and adding 66 provider calls. The variant reduced suspected mismatches while adding provider calls and verifier cost. It did not produce an accuracy figure for technical applications.

The deployment-capacity arm produced the same semantic-policy outcomes as sufficient capacity; it did not demonstrate semantic behavior under binding eviction pressure. The two policies see the same ordered requests, so a statistical comparison should use paired family-level differences. Separate overlapping intervals do not establish a significant advantage.

The common potential-reuse denominator is 800 requests whose reference answer had appeared earlier in the frozen order. Policy-specific live-cache opportunities differ because admission and capacity change which entries remain stored. The recorded p50/p95 warm HTTP timings at sufficient capacity were 35.193/51.006 ms exact, 63.894/142.934 ms static, and 211.609/364.226 ms answer-aware, each based on 999 post-first requests on one sequential local runner. These timings include local HTTP, Redis, embedding, and verifier work when active, plus controlled-provider generation on misses. They do not price or time a real cloud provider. Comparing their medians alone would hide long verifier paths and different hit/miss mixes.

6.15 Positive, Negative, and Resource Findings

The isolated 13-request release probe used an image built from commit `37dc7a7`, real Redis Stack, pinned MiniLM and NLI, required prewarm, and two mock profiles. Its seeded decorator answer was reused by Carol at rank one without provider invocation, with reported gateway latency 13.64 ms. In the same probe Alice's completed mock answer was stored in the workspace, but Carol's valid paraphrase was declined by the model; the two relevant requests took 234.64 and 277.73 ms. That is a valid-reuse miss, not an answer-quality mistake estimate. Other-organization and private requests stayed separate; an unauthorized workspace request returned 403; a secret-bearing request bypassed storage. Streaming completed and an exact repeat hit in its selected case. These are useful path checks, not a representative workload.

The 24-case authored freshness development set had 12 stable and 12 volatile requests. All 12 volatile requests bypassed, but six stable questions also bypassed. The heuristic therefore loses reuse on some technical procedure questions even before retrieval. The saved 596-token paired assertion against a 512-token verifier window produced a conservative VERIFY rather than silent truncation. That protects the decision boundary but loses a valid opportunity. Neither case should be rewritten as a model-quality success.

The Redis shortlist-read microbenchmark measured 80 rounds of hash validation: sequential median 3.357 ms, pipelined median 0.896 ms. Only those Redis reads were timed; the numbers exclude embedding, NLI, HTTP, and provider generation. Pinned NLI prewarm raised one process's resident memory by about 681 MB from import, and a 30-trial CI benchmark moved the first model-load cost into startup. These establish a local resource trade-off, not an end-to-end production speedup. Multiple workers duplicate model memory while Redis entries remain shared.

6.16 Latency and Conditional Cost Model

The JSON response's reported gateway time starts after authentication and rate admission in the handler. A browser HTTP duration begins before fetch and includes client scheduling, request transfer, server admission, response transfer, and browser processing. Streaming has a different reporting boundary. Internal stage timers isolate cache lookup, embedding, retrieval, verifier, provider, admission, and write when present. A p50 is the middle observation; p95 describes a slow-tail threshold. Neither is a universal service-level guarantee, and comparing percentiles from different datasets, cold/warm states, or provider mixes is misleading.

Let C_p denote expected provider generation cost per request, C_g the gateway's average embedding, verification, Redis, hosting and operation cost per request, and h the fraction of requests that avoid a provider call while serving a suitable answer. A simplified break-even condition is $hC_p > C_g$, assuming the gateway is an added cost and ignoring fixed migration costs. If verification also increases miss latency, the design must satisfy a latency and quality constraint in addition to that inequality. Controlled runs measure provider invocation counts but do not supply billed token totals, provider prices actually paid, or infrastructure amortization. The report therefore states avoided controlled-provider calls, not currency savings.

6.17 Testing Summary and Remaining Gates

The inspected unit, integration, and HTTP tests support specific parsing, authorization, scoped storage, completion, timeout, and failure behaviors. The selected mock HTTP probe demonstrates authorized workspace sharing and separation in a small scenario. Semantic reuse works on some controlled paraphrases and fails on others even when retrieval finds the suitable candidate. The fixed-candidate guard comparison reduced unsafe and ambiguous approvals on its authored set, while the fresh combined HTTP guard missed its promotion gate. Redis pipelining reduced the measured local shortlist-read stage, and prewarming moved cold model load into startup. Neither finding is a general provider-cost or throughput result.

Broader deployment still needs an exact-final-commit suite and browser check, independently reviewed answer-level labels, a larger family-disjoint top-k workload, realistic long answers, real-provider completion/billing smoke, privacy review, cross-worker concurrency and cancellation tests, and capacity measurements at the intended organization count. The dashboard and runtime implementation are changing concurrently; any later fix must be reconciled with its own commit and evidence before this report describes it as complete.

Chapter 7

Conclusion

7.1 Conclusion

VeriGate is an implemented organization-aware LLM gateway with exact caching by default and optional answer-aware semantic reuse. It authenticates a gateway principal, restricts workspace and private cache scopes, resolves a server-owned provider profile, checks response eligibility, and records candidate decisions and request outcomes. MiniLM and RedisSearch retrieve a shortlist; structural checks and a pinned NLI signal assess the particular stored answer before a semantic hit. A final entry read limits stale selection races, while explicit provider and admission errors avoid a fictitious successful fallback.

The saved tests support defined gateway behaviors, and the controlled evaluations expose the quality trade-off. The earlier live 25-probe run found 4/15 valid reuses despite complete shortlist retrieval. A fresh 25-probe run with combined guards reached 8/15 versus 6/15 for its baseline, with no observed invalid accepts among five cases, but it missed the predeclared promotion gate. These results justify continued experimentation, not a production answer-quality or monetary-savings claim. The work also demonstrates that similarity, question equivalence, and answer applicability are distinct decisions.

The intended product remains a shared gateway for multiple organizations, with reuse confined to each authorized workspace and private responses kept separate. Its practical value would come from avoiding repeated generation across employees while retaining a provider path for new or unsuitable requests. The current implementation establishes that workflow; the remaining work is to improve and measure the quality and economics of its reuse decisions, rather than change the product into a standalone chatbot.

7.2 Future Work

Broader deployment requires independently reviewed answer-level labels, family-disjoint calibration and holdout sets, and realistic HTTP retrieval with competing candidates. A task-specific suitability model should be trained and promoted only after its artifact, calibration, and held-out gate are fixed and verified. Further work should handle long answers and multi-step tasks, improve privacy and scope detection, test cross-worker concurrency and model memory, measure real-provider completion and billing, and validate capacity under realistic load. Those are planned studies; the current prototype remains an experimental gateway.

Appendix A

Historical Proxy Tables

The following generated tables preserve the earlier query-pair and routing experiments. Their source CSVs and scripts are historical artifacts. They do not label actual cached-answer suitability, measure billed savings, or form a before/after comparison with the answer-aware HTTP workloads in Chapter 6.

Table A.1: Candidate-pair smoke fixture. Positive recall and negative acceptance use different denominators; this is not workload hit rate or monetary savings.

System	Positive recall	Negative acceptance	Precision	Accepted
Baseline (static T=0.72)	40.0%	46.7%	46.2%	13
+ Adaptive T	33.3%	33.3%	50.0%	10
+ Verifier (Tier-1)	0.0%	0.0%	N/A	0
VeriGate (Tier-1)	0.0%	0.0%	N/A	0

Table A.2: Five-seed query-pair proxy transfer; false acceptance is among accepted pairs, not answers.

Test source	Model	Coverage	False pair acceptance
paws	gradient_boosting	64.7%	52.6%
paws	logistic_all	91.2%	55.5%
qqp	gradient_boosting	36.4%	29.8%
qqp	logistic_all	36.9%	26.6%

Table A.3: Chronological QQP pair-proxy replay; unknown cross-pair outcomes are not counted as correct.

Capacity	Policy	Known accepts	Known false	Unknown accepts
10	exact	706	0	0
10	static_0.72	722	88	2
10	adaptive_tier1	717	87	0
100	exact	925	0	0
100	static_0.72	980	145	1
100	adaptive_tier1	954	99	0

Table A.4: Synthetic IID bound exercise; reject-all runs have no defined conditional risk.

Target	Runs	Nonempty selections	True-risk violations
0.5%	50	0	0
1.0%	50	0	0
2.0%	50	35	0
5.0%	50	50	0

Table A.5: Recorded stage timings in milliseconds; these are not end-to-end provider latency.

Stage	p50	p95	p99
embed	46.51	136.69	348.71
knn_n10	0.05	0.06	1.12
knn_n100	0.05	0.06	0.13
knn_n500	0.08	0.14	0.19
knn_n1000	0.12	0.22	0.33
knn_n2000	0.34	0.71	0.90
tier1_verify	0.03	0.04	0.07
tier2_nli	364.99	882.55	1761.56
hit_tier1_n1000	29.96	47.59	61.88

Table A.6: Recorded lookup p50 latency in milliseconds; the crossover size was not measured.

Entries	In-process	RediSearch
1000	0.03	3.82
5000	0.20	4.16
10000	0.22	5.58

Table A.7: Illustrative cascade cost model. Quality retention is hard-label recall, not judged answer quality.

Threshold	Escalation	Simulated cost reduction	Hard-label recall
0.0	100.0%	0.0%	100.0%
0.35	50.0%	46.7%	100.0%
0.5	42.5%	53.7%	85.0%
0.7	17.5%	77.0%	35.0%
1.0	0.0%	93.3%	0.0%

Bibliography

[1] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), 2023.

[2] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of EMNLP*, pages 632–642, 2015.

[3] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[5] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.

[6] GPTCache Contributors. GPTCache examples and similarity evaluation. `https://github.com/zilliztech/GPTCache/blob/main/examples/README.md`. Project documentation, accessed 2026-11-01.

[7] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations (ICLR)*, 2021.

[8] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[10] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[11] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.

[12] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024.

[13] Prometheus Authors. Prometheus: Monitoring system and time series database. `https://prometheus.io`, 2024. Accessed: 2026-10.

[14] Sebastián Ramírez. FastAPI: A modern, fast web framework for building APIs with python. https://fastapi.tiangolo.com, 2024. Accessed: 2026-10.

[15] Redis Ltd. Redis stack and RediSearch: Vector similarity search. https://redis.io/docs/interact/search-and-query/advanced-concepts/vectors/, 2024. Accessed: 2026-10.

[16] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.

[17] Nils Reimers and Iryna Gurevych. Sentence-transformers: all-MiniLM-L6-v2 model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2, 2021. Accessed: 2026-10.

[18] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

[19] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. vCache: Verified semantic prompt caching. In *International Conference on Learning Representations*, 2026. https://arxiv.org/abs/2502.03771.

[20] Asmit Kumar Singh, Haozhe Wang, Laxmi Naga Santosh Attaluri, Tak Chiam, and Weihua Zhu. Asynchronous verified semantic caching for tiered LLM architectures, 2026. https://arxiv.org/abs/2602.13165.

[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

[22] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[23] Kaijie Zhu, Jindong Wang, et al. A survey on efficient inference for large language models, 2024.